

**URI – Universidade Regional Integrada do Alto Uruguai e das
Missões – Campus de Erechim**

Apostila de Compiladores

Professor Fabio A. Zanin
Fevereiro de 2026

SUMÁRIO

1	REVISÃO - CONCEITOS, LINGUAGENS E SUAS REPRESENTAÇÕES	4
1.1	TIPOS	6
1.2	ESPECIFICAÇÃO DE UMA LINGUAGEM	7
1.3	ANALISADOR LÉXICO (OU SCANNER)	9
1.3.1	<i>Divisão das Tarefas de Análise</i>	9
1.3.2	<i>Atributos para os tokens – Classe, Valor e Posição</i>	10
1.3.3	<i>Especificação de Variáveis (identificadores) e Palavras Reservadas</i>	12
2	PROCESSADORES DE LINGUAGENS	13
2.1	FERRAMENTAS PARA PROCESSAMENTO DE LINGUAGENS	15
2.2	TIPOS DE COMPILAÇÃO	16
2.2.1	<i>Implementação Compilada (ou processo de compilação)</i>	16
2.2.1.1	<i>Linkedição</i>	19
2.2.2	<i>Interpretação Pura</i>	20
2.2.3	<i>Sistemas de Interpretação Híbridos</i>	21
3	O PAPEL DO ANALISADOR SINTÁTICO	23
3.1	ANÁLISE SINTÁTICA TOP-DOWN (OU DESCENDENTE)	24
3.1.1	<i>Análise Sintática de Descendência Recursiva</i>	24
3.1.2	<i>Análise Sintática Não-Recursiva Preditiva</i>	27
3.1.3	<i>Análise Preditiva Tabular (não recursiva)</i>	28
3.1.4	<i>FIRST</i>	32
3.1.4.1	<i>FIRST - Exercícios propostos</i>	33
3.1.5	<i>FOLLOW</i>	34
3.1.5.1	<i>FOLLOW - Exercícios Propostos</i>	36
3.1.6	<i>Construção da Tabela de Parsing (Tabela de Análise)</i>	37
3.1.6.1	<i>Tabela de Parsing - Exercícios Propostos</i>	38
3.2	ANÁLISE SINTÁTICA BOTTOM-UP	40
3.2.1	<i>Análise Sintática de Precedência de Operadores</i>	41
3.2.1.1	<i>Construção da Tabela de Precedência de Operadores</i>	42
3.2.1.2	<i>Analizador Shift-Reduce para Gramáticas de Precedência de Operadores</i>	43
	<i>Exemplo:</i>	44
3.2.1.3	<i>Exercícios: Autômatos de Pilha Bottom-UP - Shift-Reduce</i>	44
3.3	TRATAMENTO DOS ERROS DE SINTAXE	45
3.3.1	<i>Estratégias de Recuperação de Erros</i>	48
3.4	ANALISADOR SEMÂNTICO	49
4	MÁQUINA DE TURING	51
4.1	UM BREVE HISTÓRICO SOBRE ALAN TURING	51
4.2	A MÁQUINA DE TURING	52
4.3	DEFINIÇÃO DE MÁQUINA DE TURING	53
4.4	DEFINIÇÃO FORMAL DA MÁQUINA DE TURING	54
4.5	LINGUAGENS RECONHECIDAS POR MÁQUINA DE TURING	55
4.6	MÁQUINA DE TURING DETERMINÍSTICA	56
4.6.1	<i>Exercícios</i>	57
5	TRANSFORMAÇÕES EM GLC'S	61
5.1	ELIMINAÇÃO DE ϵ -PRODUÇÕES (PRODUÇÕES VAZIAS)	61
5.2	ELIMINAÇÃO DE RECURSÃO À ESQUERDA	63
5.3	FATORAÇÃO À ESQUERDA	65
5.4	ELIMINANDO A AMBIGUIDADE DAS GLC'S	67
5.5	TRANSFORMAÇÕES EM GLC'S - EXERCÍCIOS PROPOSTOS	69
	BIBLIOGRAFIA	72

ÍNDICE DE FIGURAS

FIGURA 1: HIERARQUIA DE CHOMSKY	6
FIGURA 2: AUTÔMATO DO ANALISADOR LÉXICO	12
FIGURA 3: PROCESSADORES DE LINGUAGENS	13
FIGURA 4: INTERPRETADOR.	13
FIGURA 5: TRADUTOR.....	13
FIGURA 6: TIPOS DE TRADUTORES.....	14
FIGURA 7: O PRÉ-PROCESSADOR E O CROSS-COMPILER.	14
FIGURA 8: UM SISTEMA DE PROCESSAMENTO DE LINGUAGEM.	15
FIGURA 9: O PROCESSO DE COMPILAÇÃO.	17
FIGURA 10: TABELA DE SÍMBOLOS.....	18
FIGURA 11: LINKEDIÇÃO.....	19
FIGURA 12: INTERPRETAÇÃO PURA.	21
FIGURA 13: SISTEMA DE INTERPRETAÇÃO HÍBRIDO.	22
FIGURA 14: ÁRVORE DE DERIVAÇÃO	23
FIGURA 15: REDUÇÃO E EXPANSÃO.....	24
FIGURA 16: ETAPAS NUMA ANÁLISE SINTÁTICA TOP-DOWN.	26
FIGURA 17: ANALISADOR PREDITIVO TABULAR.	28
FIGURA 18: FITA E UNIDADE DE CONTROLE DE UMA MÁQUINA DE TURING.....	54
FIGURA 19: REPRESENTAÇÃO DA FUNÇÃO PROGRAMA COMO UM GRAFO.....	55
FIGURA 20: MÁQUINA DE TURING	56
FIGURA 21: SEQUÊNCIA DE PROCESSAMENTO DE UMA MÁQUINA DE TURING	57
FIGURA 22: ÁRVORE GRAMATICAL PARA O ENUNCIADO CONDICIONAL [AHO].	67
FIGURA 23: DUAS ÁRVORES GRAMATICAIIS PARA UMA SENTENÇA AMBÍGUA [AHO].	68

1 REVISÃO - CONCEITOS, LINGUAGENS E SUAS REPRESENTAÇÕES

Segundo MENEZES (1997), os estudos sobre linguagens formais iniciaram-se na década de 50 com o objetivo de definir matematicamente as estruturas das linguagens naturais. No entanto, verificou-se que a teoria desenvolvida se aplicava ao estudo das linguagens de programação.

A teoria de linguagens formais engloba, basicamente, o estudo das características, propriedades e aplicações das linguagens formais, bem com a forma de representação da estrutura (sintaxe) e determinação do significado (semântica) das sentenças das linguagens. Segundo FURTADO (1992), a importância desta teoria é dupla: tanto apoia outros aspectos teóricos da teoria da computação, tais como decibilidade, computabilidade, complexidade, etc. como fundamenta diversas aplicações computacionais, como, por exemplo, processamento de linguagens, reconhecimento de padrões, modelagem de sistemas, etc., mas, o que é uma linguagem?

- Uma linguagem é uma forma de comunicação, usada por sujeitos de uma determinada comunidade;
- Uma linguagem é o conjunto de SÍMBOLOS e REGRAS para combinar esses símbolos em sentenças sintaticamente corretas.
- "uma linguagem é formal quando pode ser representada através de um sistema com sustentação matemática" (PRICE; EDELWEISS, 1989).

Assim sendo, são necessários conceitos matemáticos para o estudo das linguagens formais.

- **Símbolo**

Um símbolo é uma entidade abstrata básica sem definição formal. São exemplos de símbolos as letras, os dígitos, etc. Símbolos são ordenáveis lexicograficamente e, portanto, podem ser comparados quanto à igualdade ou precedência. Por exemplo, tomando as letras dos alfabetos, tem-se a ordenação $A < B < C < \dots < Z$. A principal utilidade dos símbolos está na possibilidade de usá-los como elementos atômicos em definições de linguagens.

- **Sentença (ou palavra)**

Uma sentença (ou palavra) é uma sequência finita de símbolos. Sejam P , R , I , M , e A símbolos, então $PRIMA$ é uma sentença. As sentenças vazias, representadas por ϵ , são constituídas por nenhum símbolo.

- **Tamanho de uma sentença**

O tamanho (ou comprimento) de uma sentença w , denotado por $|w|$, é dado pelo número de símbolos que compõem w . Assim, o tamanho da sentença 'LÁPIS' é 5 e o tamanho da sentença vazia é 0.

- **Alfabeto**

Um alfabeto, denotado por V , é um conjunto finito de símbolos. Assim, considerando os símbolos dígitos, letras, etc., tem-se os seguintes alfabetos $V_{\text{binário}} = \{0, 1\}$; $V_{\text{vogais}} = \{a, e, i, o, u\}$. É importante observar que um conjunto vazio também pode ser considerado um alfabeto.

O fechamento reflexivo de um alfabeto V , denotado por V^* , é o conjunto infinito de todas as sentenças que podem ser formadas com os símbolos de V , inclusive a sentença vazia (ou sentença nula). O fechamento transitivo de um alfabeto V , denotado por V^+ , é dado por $V^* - \{\epsilon\}$. Seja V o alfabeto dos dígitos binários, $V = \{0, 1\}$. O fechamento transitivo de V é $V^+ = \{0, 1, 00, 01, 10, 11, 000, 001, 010, 011, \dots\}$, enquanto que o fechamento reflexivo de V é $V^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 011, \dots\}$.

- **Linguagens e suas Representações**

Uma linguagem formal L é um conjunto de sentenças formadas por símbolos tomados de algum alfabeto V . Isto é, uma linguagem sobre o alfabeto V é um subconjunto de V^* ($L \subseteq V^*$). Assim, por exemplo, o conjunto de sentenças válidas da língua portuguesa poderia ser definido extensionalmente como um subconjunto de $\{a, b, c, \dots, z\}^+$.

Uma linguagem pode ser:

- Finita: quando é composta por um conjunto finito de sentenças. Seja $V = \{a, b\}$ um alfabeto, L_1 , L_2 e L_3 linguagens definidas conforme segue:
 $L_1 = \{w \mid w \in V^* \wedge |w| < 3\}$, ou seja, L_1 é uma linguagem constituída por todas as sentenças de tamanho menor que 3 formadas por símbolos de V . Portanto, $L_1 = \{\epsilon, a, b, aa, ab, ba, bb\}$
 $L_2 = \emptyset$
 $L_3 = \{\epsilon\}$

As linguagens L_2 e L_3 são diferentes.

- Infinita: quando é composta por um conjunto infinito de sentenças. Seja $V = \{a, b\}$ um alfabeto, L_1 , L_2 e L_3 linguagens definidas conforme segue:
 $L_1 = \{w \mid w \in V^* \wedge |w| \text{ MOD } 2 = 0\}$, ou seja, L_1 é uma linguagem constituída por todas as sentenças de tamanho par formadas por símbolos de V . Portanto, $L_1 = \{\epsilon, aa, ab, ba, bb, aaaa, aaab, aaba, \dots\}$

$L_2 = \{w \mid w \text{ é um palíndromo}^2\}$. Portanto $L_2 = \{\epsilon, a, b, aa, bb, aba, baab, \dots\}$

$L_3 =$ linguagem de programação PASCAL, ou seja, o conjunto infinito de programas escritos (combinações infinitas) na linguagem de programação em questão.

Como definir/representar uma linguagem? Uma linguagem finita pode ser definida através da enumeração das sentenças constituintes ou através de uma descrição algébrica (por extensão). Uma linguagem infinita deve ser definida através de representação finita. Reconhecedores ou sistemas geradores são dois tipos de representações finitas. Um reconhecedor é um dispositivo formal usado para verificar se uma determinada sentença pertence ou não à linguagem. São exemplos de reconhecedores autômatos finitos, autômatos de pilha e máquinas de Turing. Um sistema gerador é um dispositivo formal usado para gerar de forma sistemática as sentenças de uma dada linguagem. São exemplos de sistemas geradores as gramáticas.

Todo reconhecedor e todo sistema gerador pode ser representado por um algoritmo.

¹ MOD é uma operação que representa o resto da divisão inteira.

² Um palíndromo é uma palavra que tem a mesma leitura da esquerda para a direita e vice-versa.

1.1 TIPOS

Noam Chomsky definiu uma hierarquia de linguagens como modelos para linguagens naturais. As classes de linguagens são: regular, livre de contexto, sensível ao contexto e irrestrita. As inclusões dos tipos de linguagens são apresentadas na figura abaixo.

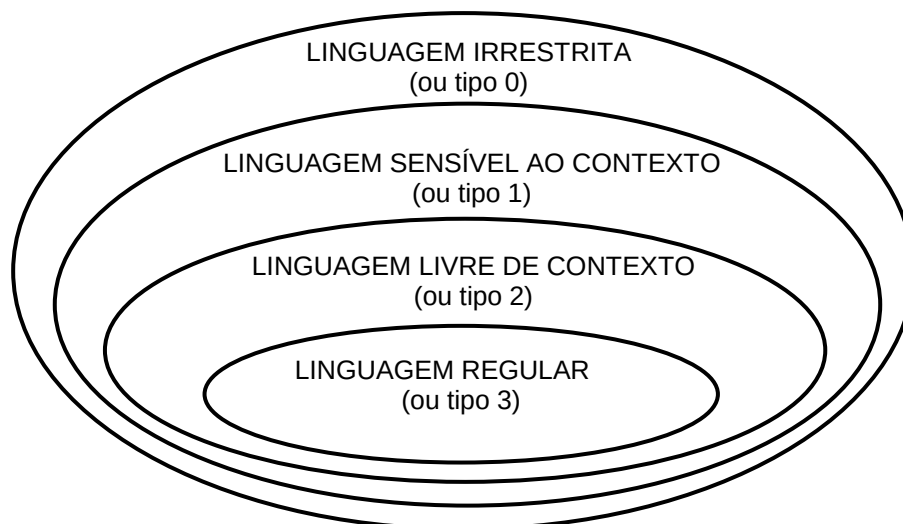


FIGURA 1: HIERARQUIA DE CHOMSKY

Pode-se ver que uma linguagem do tipo 3 é também do tipo 2, uma linguagem do tipo 2 é também do tipo 1, e uma linguagem do tipo 1 é também do tipo 0. Estas inclusões são estritas, isto é, existem linguagens do tipo 0 que não são do tipo 1, existem linguagens do tipo 1 que não são do tipo 2 e existem linguagens do tipo 2 que não são do tipo 3.

As **linguagens regulares** constituem um conjunto de linguagens bastante simples. Essas linguagens podem ser reconhecidas por autômatos finitos, geradas por gramáticas regulares e facilmente descritas por expressões simples, chamadas expressões regulares. Segundo MENEZES (1997), algoritmos para reconhecimento e geração de linguagens regulares são de grande eficiência, fácil implementação e pouca complexidade. HOPCROFT; ULLMAN (1979) afirmam que vários são os problemas cujo desenvolvimento pode ser facilitado pela conversão da especificação feita em termos de expressões regulares na implementação de um autômato finito correspondente. Dentre as várias aplicações, podemos citar: analísadores léxicos e editores de texto. Para descrever precisamente a regra de formação (padrão) de *tokens* mais complexos de linguagens de programação, tais como identificadores, palavras reservadas, constantes e comentários, usa-se a notação das expressões regulares, as quais podem ser automaticamente convertidas em autômatos finitos equivalentes cuja implementação é trivial. A operação de busca e substituição de palavras (ou trechos de palavras) também pode ser realizada através da implementação de um autômato finito a partir da especificação da expressão regular correspondente.

A importância das **linguagens livres de contexto** reside no fato de que especificam adequadamente as estruturas sintáticas das linguagens de programação, tais como parênteses balanceados, construções aninhadas, entre outras. A maioria das linguagens de programação pertence ao conjunto das linguagens livre de contexto e pode ser analisada por algoritmos eficientes. Essas linguagens podem ser reconhecidas por autômatos de pilha e geradas por gramáticas livre de contexto (GLC). Segundo FURTADO (1992), dentre as várias aplicações dos conceitos de linguagens livre de contexto, podemos citar: definição e especificação de linguagens de programação; implementação eficiente de analísadores sintáticos; implementação de tradutores de linguagens e processadores de texto em geral; estruturação formal e análise computacional de linguagens naturais.

Segundo MENEZES (1997), as **linguagens sensíveis ao contexto e irrestritas** "permitem explorar os limites da capacidade de desenvolvimento de reconhecedores ou geradores de linguagens, ou seja, estuda a solucionabilidade do problema da existência de algum reconhecedor ou gerador para determinada linguagem". Essas linguagens podem ser respectivamente reconhecidas por *máquinas de Turing limitadas* e geradas por *gramáticas sensíveis ao contexto* (GSC); reconhecidas por *máquinas de Turing* e geradas por *gramáticas irrestritas*.

MENEZES (1997) observa que nem sempre as linguagens de programação são tratadas adequadamente na hierarquia de Chomsky. Assim, para a especificação de determinadas linguagens de programação pode-se fazer necessário o uso de outros formalismos como por exemplo gramática de grafos.

1.2 ESPECIFICAÇÃO DE UMA LINGUAGEM

Foi dito que uma linguagem L é qualquer subconjunto de sentenças sobre um alfabeto V . Mas, qual subconjunto é esse, como defini-lo? Uma gramática é um dispositivo formal usado para definir qual subconjunto de V^* forma determinada linguagem. A gramática define uma estrutura sobre um alfabeto de forma a permitir que apenas determinadas combinações de símbolos sejam consideradas sentenças.

- **Gramática**

O que é GRAMÁTICA? É um sistema gerador de linguagens; é um sistema de reescrita; é uma maneira finita de descrever uma linguagem; é um dispositivo formal usado para especificar de maneira finita e precisa uma linguagem infinita.

Formalmente uma gramática G é definida como sendo uma quádrupla $G = (N, T, P, S)$, onde: N é um conjunto finito de símbolos denominados símbolos não-terminais, usados na descrição da linguagem; T é um conjunto finito de símbolos denominados símbolos terminais, os quais são os símbolos propriamente ditos; P é conjunto finito de pares (α, β) denominados regras de produção (ou regras gramaticais) que relacionam os símbolos terminais e não-terminais; S é o símbolo inicial da gramática pertencente a N , a partir do qual as sentenças de uma linguagem podem ser geradas.

As regras gramaticais são representadas por $\alpha ::= \beta$ ou $\alpha \rightarrow \beta$, onde α e β são sentenças sobre V , com α envolvendo pelo menos um símbolo pertencente a N . Uma sequência de regras de produção da forma $\alpha \rightarrow \beta_1, \alpha \rightarrow \beta_2, \dots, \alpha \rightarrow \beta_n$, ou seja, com a mesma componente do lado esquerdo, pode ser abreviada como uma única produção na forma: $\alpha \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$. O significado de uma regra de produção $\alpha \rightarrow \beta$ é α produz β ou α é definido por β .

Abaixo se encontra um exemplo de uma gramática:

EXEMPLO 1: a linguagem dos números inteiros sem sinal é gerada pela seguinte gramática G:

$G = (N, T, P, S)$
onde
 $V_N = \{N, D\}$
 $V_T = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 $P = \{$
 $N \rightarrow D N \mid D$
 $D \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$
 $\}$
 $S = N$

Observa-se que a gramática apresentada no EXEMPLO 1 não é regular.

- **Gramáticas Regulares (tipo 3)**

Uma gramática G é de tipo 3, ou *regular*, se cada produção é da forma:

$A ::= aB,$
 $A ::= a$ ou
 $A ::= \varepsilon,$ onde A e B são não-terminais e a é um terminal.

Exemplo de Gramática Regular:

$S ::= 0A \mid 1A$
 $A ::= 0B \mid 1A$
 $B ::= 0A \mid 1B \mid \varepsilon$

- **Gramáticas Livres de Contexto - GLC (tipo 2)**

As linguagens livres de contexto são definidas por **gramáticas livres de contexto** (GLC) que podem ser utilizadas para especificar a parte sintática de uma linguagem de programação. Uma gramática livre de contexto é uma quádrupla $G = (N, T, P, S)$, onde: $P = \{A \rightarrow \alpha \mid A \in N, \alpha \text{ é uma sentença em } (N \cup T)^*\}$. Em outras palavras, uma gramática livre de contexto admite apenas regras de produção cujo lado esquerdo contém exatamente um não-terminal. Segundo MENEZES (1997), significa que o não-terminal "A deriva α **sem depender ('livre')** de qualquer análise dos símbolos que antecedem ou sucedem A ('contexto') na sentença que está sendo derivada".

EXEMPLO: a linguagem $L = \{x \mid x \in \{a, b\}^+ \wedge x \text{ segue o padrão de formação } a^n b^n \wedge n \geq 0\}$ é gerada pela seguinte gramática livre de contexto:

$S \rightarrow a S b \mid \varepsilon$

Pode-se estabelecer uma analogia entre esta linguagem e os blocos estruturados do tipo *BEGIN END*, ou as expressões com parênteses balanceados na forma $(^n)^n$.

- **Gramáticas Sensíveis ao Contexto - GSC (tipo 1)**

1. O Símbolo inicial **S** não aparece no lado direito de nenhuma produção, e
2. Para cada produção $\alpha_1 ::= \alpha_2$, é verdade que $|\alpha_1| \leq |\alpha_2|$, ou seja, o lado esquerdo da produção é $\leq$ ao lado direito (com exceção da regra $S ::= \epsilon$), então se diz que **G** é uma gramática do tipo 1, ou *Gramática Sensível ao Contexto*.

$S \rightarrow A A A \mid a A$
 $A A \rightarrow A b c A A$
 $A \rightarrow a \mid b$

- **Linguagens Irrestritas (tipo 0)**

As linguagens irrestritas representam todas as linguagens que podem ser reconhecidas mecanicamente e em um tempo finito. São definidas por **gramáticas irrestritas** (GI). Uma gramática irrestrita é uma quádrupla $G = (N, T, P, S)$ que não possui qualquer restrição quanto à forma das regras de produção. Qualquer uma das gramáticas especificadas anteriormente é um exemplo de uma gramática irrestrita.

Observa-se que uma linguagem com dois ou mais símbolos terminais que possuem uma relação quantitativa e posicional não pode ser representada por GR; uma linguagem com três ou mais símbolos terminais que possuem uma relação quantitativa e posicional não pode ser representada por GLC; quanto mais abrangente o tipo da gramática mais complexa é a análise.

1.3 ANALISADOR LÉXICO (OU SCANNER)

Esta seção trata das técnicas para especificar analisadores léxicos. Uma forma simples de se construir um analisador léxico é escrever um diagrama que ilustre a estrutura dos tokens da linguagem-fonte e então traduzi-lo manualmente num programa que os localize. Analisadores léxicos eficientes podem ser produzidos dessa forma [AHO].

Para AHO, as técnicas usadas para implementar os analisadores léxicos podem também ser aplicadas a outras áreas. Em cada aplicação, o problema subjacente é a especificação e o projeto de programas que executem ações disparadas por padrões nas cadeias de caracteres.

Várias linguagens usam expressões regulares para descrever padrões. Por exemplo, o prompt de linha de comando de um sistema operacional qualquer, permite que o usuário se refira a um conjunto de nomes de arquivos escrevendo uma expressão regular. No Linux, o comando `rm -i *.txt`, por exemplo, solicita confirmação antes de remover todos os arquivos de texto (terminados em ".txt") da pasta atual.

1.3.1 Divisão das Tarefas de Análise

Segundo [AHO], existem várias razões para se dividir a fase de análise da compilação em análise léxica e análise sintática ou gramatical (*parsing*):

a) Um projeto mais simples talvez seja a consideração mais importante. A separação das análises léxica e sintática frequentemente nos permite simplificar uma ou outra dessas fases. Por exemplo, um parser que incorpore as convenções para comentários e espaços em branco é significativamente mais complexo do que um que assuma que os mesmos já tenham sido

removidos pelo analisador léxico. Se estivermos projetando uma nova linguagem, separar as convenções léxicas das sintáticas pode levar a um projeto global de linguagem mais claro.

b) A eficiência do compilador é melhorada. Um analisador léxico separado nos permite construir um processador especializado e potencialmente mais eficiente para a tarefa. Uma grande quantidade de tempo é gasta lendo-se o programa-fonte e particionando-o em tokens. Técnicas de buferização especializadas para a leitura de caracteres e o processamento de tokens podem acelerar significativamente o desempenho de um compilador.

c) A portabilidade do compilador é realçada. As peculiaridades do alfabeto de entrada e outras anomalias específicas de dispositivos podem ser restringidas ao analisador léxico. A representação de símbolos especiais ou não-padrão, tais como ↑ em Pascal, podem ser isoladas no analisador léxico.

1.3.2 Atributos para os tokens – Classe, Valor e Posição

Conforme mencionado anteriormente, a função do analisador léxico é ler uma sequência de caracteres que constitui um programa fonte e coletar, dessa sequência, os tokens (palavras de uma linguagem regular) que constituem o programa. Os *tokens* ou símbolos léxicos são as unidades básicas do texto do programa. Cada *token* é representado internamente por três informações:

1. **Classe do token**, que representa o tipo do *token* reconhecido. Exemplos de classes são: identificadores, constantes numéricas, cadeias de caracteres, palavras reservadas, operadores e separadores.
2. **Valor do Token**, o qual depende da classe. Para *tokens* da classe constante inteira, por exemplo, o valor do *token* pode ser o número inteiro representado pela constante. Para *tokens* da classe identificador, o valor pode ser a sequência de caracteres, lida no programa fonte, que representa o identificador, ou um apontador para a entrada da Tabela de símbolos. Algumas classes de tokens, como as palavras reservadas, não têm valor associado.
3. **Posição do token**, a qual indica o local do texto fonte (linha e coluna) onde ocorreu o token. Essa informação é utilizada, principalmente, para indicar o local de erros.

Em função do campo *valor*, os tokens podem ser divididos em dois grupos:

- *Tokens simples*, que são os tokens que não têm valor associado (como as palavras reservadas, operadores e delimitadores) porque a classe do *token* descreve-o completamente. Esses *tokens* correspondem a elementos fixos da linguagem.
- *Tokens com argumento*, que são tokens que têm um valor associado (como identificadores e constantes). Correspondem aos elementos da linguagem definidos pelo programador como, por exemplo, identificadores, constantes numéricas e cadeias de caracteres.

A seguir é repetida a sequência de *tokens* que é obtida do trecho de programa abaixo:

1	2	3	4	5	6	7	8	9	10	11	12	13	14
W	H	I	L	E		I	<	1	0	0		D	O
				I	:	=	J	+	I	;			

```
While I < 100 do
    I := J + I;
```

A análise léxica desse segmento produz a seguinte sequência:

[whi, 1] [id, 7] [<, 8] [cte, 9] [do, 13]

[id, 5] [:=, 6] [id, 8] [+ , 9] [id, 10] [;, 11]

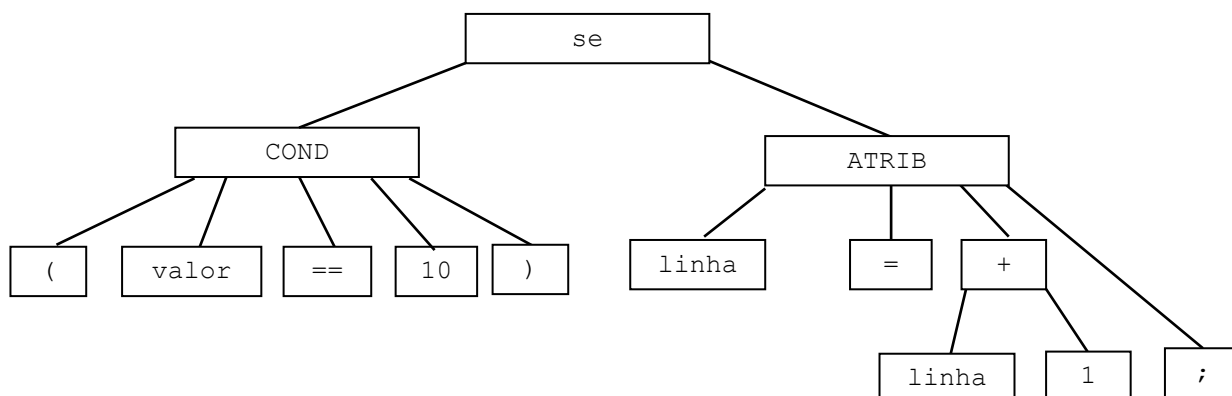
Para simplificar, os tokens estão representados por pares e são relacionados conforme a linha em que estão no programa-fonte. Eles são representados pelo par [classe_token, índice_tabela]. As classes para palavras reservadas constituem-se em abreviações dessas, não sendo necessário passar seus valores para o analisador sintático. Para delimitadores e operadores, a classe é o próprio valor do token. Usualmente, os compiladores representam a classe de um token por um número inteiro para tornar a representação mais compacta. Neste texto, empregou-se uma representação simbólica para ajudar a compreensão.

```
se (valor == 10)
    linha = linha + 1;
```

Análise Léxica

1 <se, PR> < (> <valor, ID> <==, RL> <10, int> <) > <linha, ID> <=, ATR> <linha, ID> < + > <1, int> < ; , FC>

Análise Sintática



Linha	Item Léxico	Classe
1	se	PR
1	(	(
1	valor	ID
1	==	RL
1	10	Int
1	)	)
1	linha	ID
1	=	ATR
1	linha	ID
1	+	+
1	1	int
1	;	FC

1.3.3 Especificação de Variáveis (identificadores) e Palavras Reservadas

Na maioria das linguagens de programação, as palavras reservadas da linguagem são casos particulares dos identificadores e seguem a mesma notação desses. Logo, as especificações de analisadores léxicos, usualmente, não expressam, explicitamente, o reconhecimento de palavras reservadas.

Essas palavras são armazenadas em uma tabela interna, que é examinada cada vez que um identificador é reconhecido. Se o identificador ocorre na tabela, então se trata de uma palavra reservada. Caso contrário, trata-se de um identificador.

Um exemplo do reconhecimento de tokens é apresentado abaixo:

Considere uma linguagem simples cujos tokens são os seguintes:

- Identificadores formados por uma letra seguida, opcionalmente, por uma ou mais letras e/ou dígitos;
- Números inteiros formados por um ou mais dígitos;
- Comentários delimitados por (*) e (*).

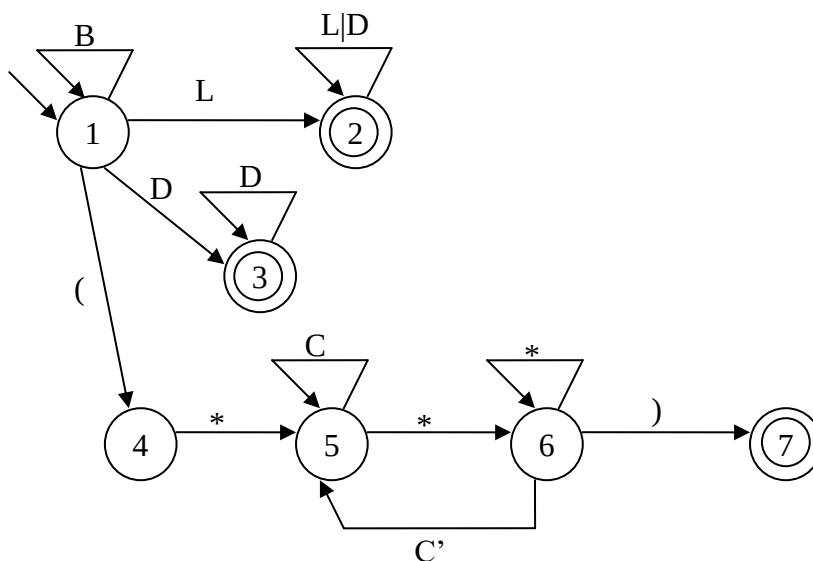


FIGURA 2: AUTÔMATO DO ANALISADOR LÉXICO

Espaços em branco são ignorados. A figura 2 representa um autômato finito que especifica o analisador léxico para essa linguagem. Nesse autômato, o símbolo B representa um espaço, os símbolos L e D representam, respectivamente, letra e dígito, e C e C' representam, respectivamente, qualquer caractere diferente de * e de * ou). Observe que há um estado terminal para cada classe de token.

2 PROCESSADORES DE LINGUAGENS

Um **Processador** é um programa que permite ao computador “entender” os comandos de alto nível escritos pelos usuários. Existem dois tipos principais de processadores de linguagem: os **Interpretadores** e os **Tradutores**, que seguem o seguinte esquema:

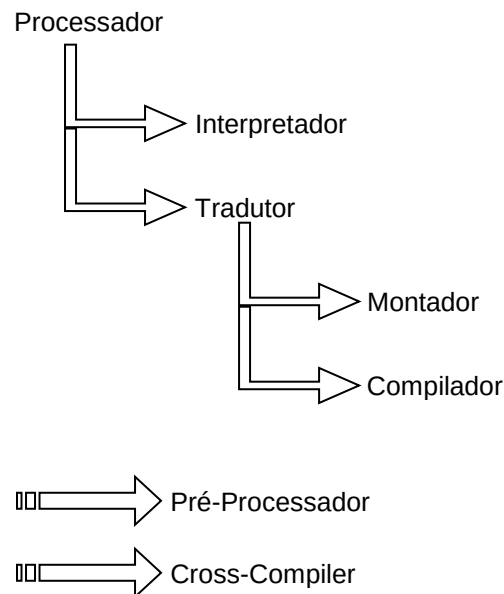


FIGURA 3: PROCESSADORES DE LINGUAGENS

Um **Interpretador** (figura abaixo) é um programa que aceita como entrada um programa escrito em uma linguagem chamada *linguagem fonte* e executa diretamente as instruções dadas nesta linguagem.



FIGURA 4: INTERPRETADOR.

Um **Tradutor** (figura abaixo) é um programa que aceita como entrada um programa escrito em uma *linguagem fonte* e produz como saída um programa escrito em uma *linguagem objeto*. Muitas vezes a *linguagem objeto* é a própria linguagem de máquina do computador. Nesse caso, o *programa objeto* pode ser diretamente executado pela máquina.



FIGURA 5: TRADUTOR.

Tradutores são divididos em dois tipos: **Montadores** e **Compiladores**, os quais traduzem linguagens de baixo nível e linguagens de alto nível, respectivamente. A figura esquematiza os dois tipos de tradutores.

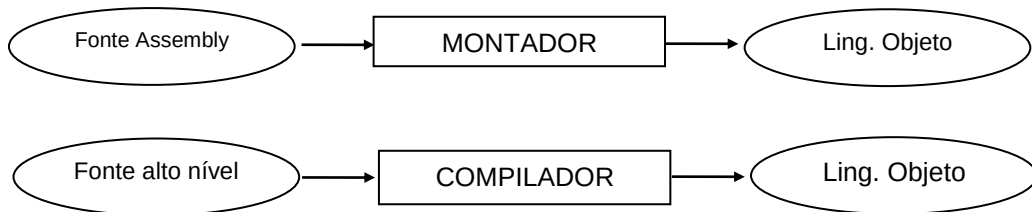


FIGURA 6: TIPOS DE TRADUTORES.

OBS: Linguagem Objeto geralmente é a própria linguagem de máquina.

Existem também o **Pré-processador** que traduz uma linguagem de alto nível em outra linguagem de alto nível e o **Cross-compiler** que gera código para outra máquina diferente da utilizada para compilação. A figura que segue esquematiza o pré-processador.

Exemplo de Cross-compiler: Compilar no desktop para rodar no móvel.

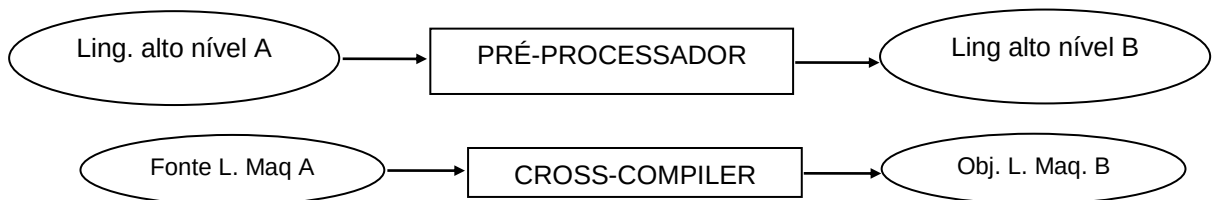


FIGURA 7: O PRÉ-PROCESSADOR E O CROSS-COMPILER.

No processamento de linguagens pode ser necessário o uso de vários processadores para traduzir um *programa fonte* composto por módulos em um *programa objeto*. A figura abaixo apresenta o exemplo de um **Sistema de Processamento de Linguagem**.

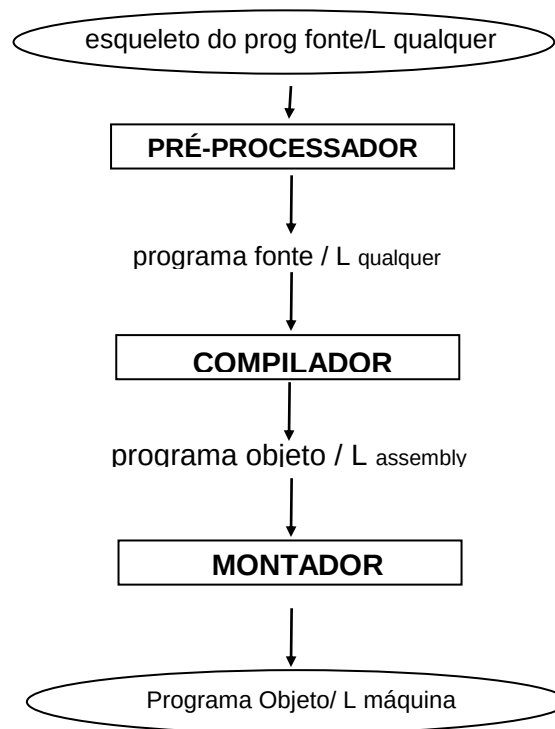


FIGURA 8: UM SISTEMA DE PROCESSAMENTO DE LINGUAGEM.

2.1 FERRAMENTAS PARA PROCESSAMENTO DE LINGUAGENS

A construção de compiladores engloba várias áreas desde a teoria de linguagens de programação até engenharia de *software*, passando por arquitetura de máquina, sistemas operacionais e algoritmos. Algumas técnicas básicas de construção de compiladores podem ser usadas na construção de ferramentas variadas para o processamento de linguagens, como por exemplo:

- **Compiladores para linguagens de programação:** um compilador traduz um programa escrito numa *linguagem fonte* em um programa escrito em uma *linguagem objeto* (entendível pela máquina);
- **Formatadores de texto:** um formatador de texto manipula um conjunto de caracteres composto pelo documento a ser formatado e por comandos de formatação (parágrafos, figuras, fórmulas matemáticas, **negrito**, *itálico*, etc). São exemplos de textos formatados os documentos escritos em editores convencionais, os documentos escritos em HTML (*HyperText Markup Language*), etc.;
- **Interpretadores de *queries* (consultas a banco de dados):** um interpretador de *queries* traduz uma *query* composta por operadores lógicos ou relacionais em comandos para percorrer um banco de dados.

2.2 TIPOS DE COMPILAÇÃO

A seguir nesta seção, vamos estudar os tipos principais de compilação. Veja:

2.2.1 Implementação Compilada (ou processo de compilação)

Os programas podem ser traduzidos para linguagem de máquina, a qual pode ser executada diretamente no computador. Isso é chamado de implementação compilada. Esse método tem a vantagem de uma execução de programa muito rápida, assim que o processo de tradução for concluído. Grande parte das implementações de linguagens de programação como o C por exemplo, dá-se por meio de compiladores.

A linguagem que um compilador traduz é chamada de linguagem-fonte. O processo de compilação desenvolve-se em diversas etapas, a maioria das quais mostrada na figura a seguir.

O analisador léxico reúne os caracteres do programa-fonte em unidades léxicas que são os identificadores, as palavras especiais, os operadores e os símbolos de pontuação. Ele ignora os comentários no programa-fonte, porque eles não têm nenhuma utilidade para o compilador.

O analisador sintático pega as unidades do analisador léxico e usa-as para construir estruturas hierárquicas chamadas árvores de análise, as quais representam a estrutura sintática do programa (árvores de derivação, lembra?). Em muitos casos, nenhuma estrutura de árvore de análise real é construída; ao contrário, a informação que seria necessária para construí-la é gerada e usada.

O gerador de código intermediário produz um programa em uma linguagem diferente, no nível intermediário entre o programa-fonte e a saída final do compilador, o programa em linguagem de máquina (note que as palavras *linguagem* e *código* muitas vezes são usadas de maneira intercambiável).

As linguagens intermediárias, às vezes, parecem-se muito com as linguagens *assembly*. Em outros casos, o código intermediário está em um nível bem mais alto que o *assembly*.

O analisador semântico faz parte integralmente do gerador de código intermediário e verifica se há erros difíceis, se não impossíveis, de serem detectados durante a análise sintática, como por exemplo, erros de tipo.

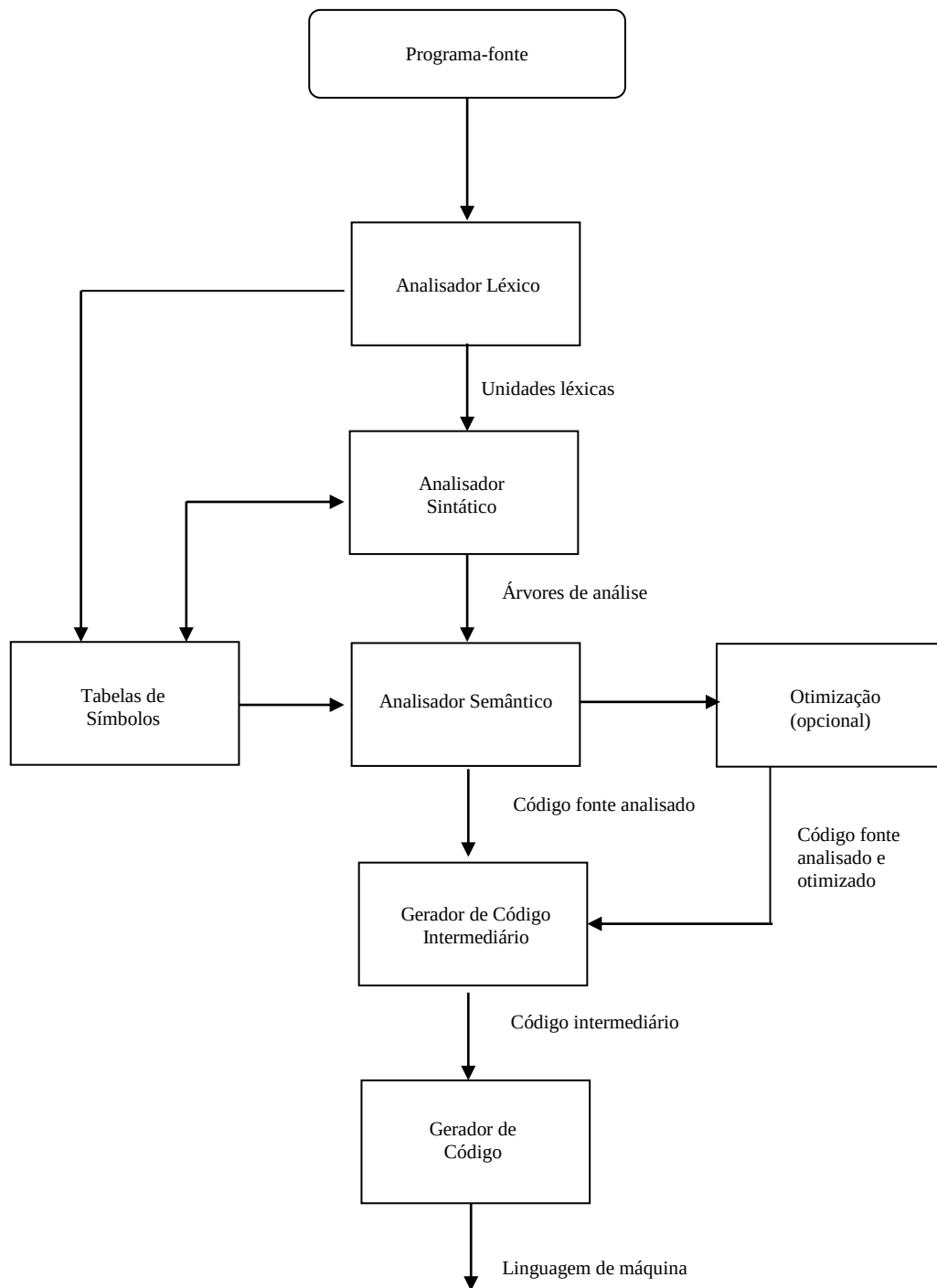


FIGURA 9: O PROCESSO DE COMPILAÇÃO.

A otimização, que melhora os programas tornando-os menores ou mais rápidos, ou ambos, muitas vezes, é uma parte opcional da compilação. De fato, alguns compiladores são incapazes de fazer qualquer otimização significativa.

Esse tipo de compilador seria usado em situações em que a velocidade de execução do programa traduzido é bem menos importante do que a velocidade de compilação. Um exemplo dessa situação é um laboratório de computação para programadores principiantes. Na maioria das situações comerciais e industriais, a velocidade de execução é mais importante do que a velocidade de compilação, de modo que a otimização é rotineiramente desejável. Uma vez que muitos tipos de otimização não podem ser feitos em linguagem de máquina, a maioria das otimizações é feita no código intermediário.

O gerador de código converte a versão do código intermediário otimizado do programa para um programa em linguagem de máquina equivalente.

A tabela de símbolos serve como um banco de dados para o processo de compilação. Seu principal conteúdo são informações sobre tipos e atributos de cada nome definido pelo usuário no programa. Essas informações são colocadas na tabela de símbolos pelos analisadores léxico e sintático e usadas pelo analisador semântico e pelo gerador de código. Na figura abaixo segue um exemplo de uma tabela de símbolos gerada por um analisador léxico de um compilador:

Linha	Item Léxico	Classe
1	programa	PR
1	demo	ID
1	;	FC
2	variaveis	PR
3	A	ID
3	,	VG
3	B	ID
3	:	DV
3	inteiro	PR
3	;	FC
4	inicio	PR
5	A	ID
5	:=	AV
5	0	NU
5	;	FC
6	b	ID
6	:=	AV
6	a	ID
6	+	OP
6	5	NU
6	;	FC
7	fim	PR
7	.	PF

FIGURA 10: TABELA DE SÍMBOLOS.

2.2.1.1 Linkedição

Conforme se afirmou anteriormente, não obstante a linguagem de máquina gerada por um compilador possa ser executada diretamente no hardware, quase sempre ela deve ser executada juntamente com algum outro código.

A maioria dos programas de usuário também exige programas do sistema operacional. Entre os mais comuns, estão aqueles para entrada (input) e saída (output) de dados. O compilador cria chamadas a programas do sistema necessários quando o programa de usuário necessita deles. Antes que os programas em linguagem de máquina produzidos pelo compilador possam ser executados, os programas necessários do sistema operacional devem ser encontrados e vinculados ao usuário.

A operação de vinculação conecta o programa de usuário aos de sistema, colocando os endereços dos pontos de entrada dos programas de sistema nas chamadas a eles no de usuário. O código de usuário e o de sistemas juntos, às vezes, são chamados de módulo de carga ou imagem de executável. O processo de coletar programas de sistema e vinculá-los aos programas de usuário é chamado de vinculação e carregamento ou, às vezes, apenas de vinculação. Ele é realizado por um programa de sistema chamado linkeditor, conforme a figura que segue.

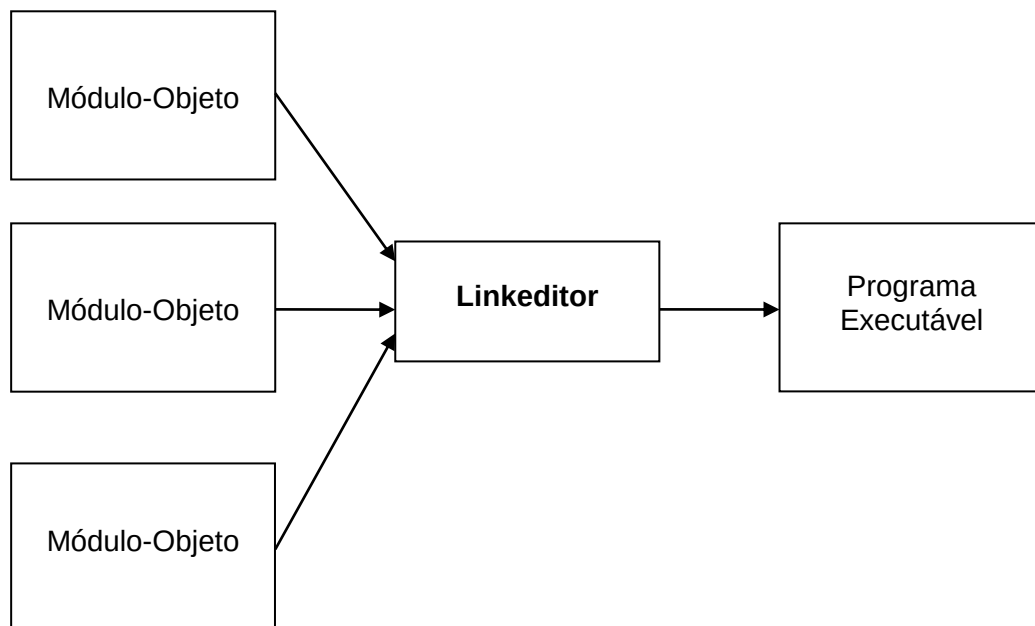


FIGURA 11: LINKEDIÇÃO.

Além dos programas de sistema, os programas de usuário muitas vezes devem ser vinculados a programas de usuário compilados anteriormente, que residem em **bibliotecas**. Assim, o *linkeditor* não somente vincula algum dado programa aos programas de sistema, mas também o vincula a outros programas de usuário.

2.2.2 Interpretação Pura

Na extremidade oposta dos métodos de implementação, os programas podem ser interpretados por outro programa chamado **interpretador**, sem nenhuma conversão. O programa interpretador age como uma simulação de software de uma máquina cujo ciclo buscar-executar lida com instruções de programa em linguagem de alto nível em vez de instruções de máquina. Essa simulação de software, evidentemente, fornece uma máquina virtual para a linguagem.

Essa técnica, chamada de **interpretação pura** ou, simplesmente de **interpretação**, tem a vantagem de permitir uma fácil implementação de muitas operações de depuração do código-fonte, porque todas as mensagens de erro em tempo de execução podem referir-se a unidades do código. Por exemplo, se for considerado que um índice de *array* está fora da faixa, a mensagem de erro poderá facilmente indicar a linha do fonte e o nome do *array*.

Por outro lado, esse método tem a séria desvantagem de que a execução é de 10 a 100 vezes mais lenta que em sistemas compilados. A principal causa dessa lentidão é a decodificação das instruções de alto nível, bem mais complexas do que as instruções em linguagem de máquina (não obstante possa haver um número menor de comandos do que de instruções de código de máquina equivalente). Portanto, a decodificação de comandos, em vez da conexão entre o processador e a memória, é o gargalo de um interpretador puro.

Outra desvantagem da interpretação pura é que ela frequentemente exige mais espaço. Além do programa-fonte, a tabela de símbolos deve estar presente na interpretação. Além disso, o programa-fonte deve ser armazenado em uma forma projetada para permitir fácil acesso e modificação, em vez de um tamanho mínimo.

A interpretação é um processo difícil em programas escritos em uma linguagem complicada, porque o significado de cada expressão e instrução deve ser determinado diretamente do programa-fonte em tempo de execução. Linguagens com estruturas mais simples prestam-se à interpretação pura. Por exemplo, PHP e HTML, às vezes são implementadas como sistemas interpretativos puros.

A maioria dos comandos dos sistemas operacionais modernos, como os scripts de shell no Linux (usando bash, zsh, entre outros) ou os arquivos de script no PowerShell do Windows, é executada por interpretadores puros. Linguagens de programação mais complexas e amplamente utilizadas atualmente, como C++ e Java, raramente são implementadas com interpretadores puros, sendo geralmente compiladas. O processo de interpretação pura é ilustrado na figura a seguir.

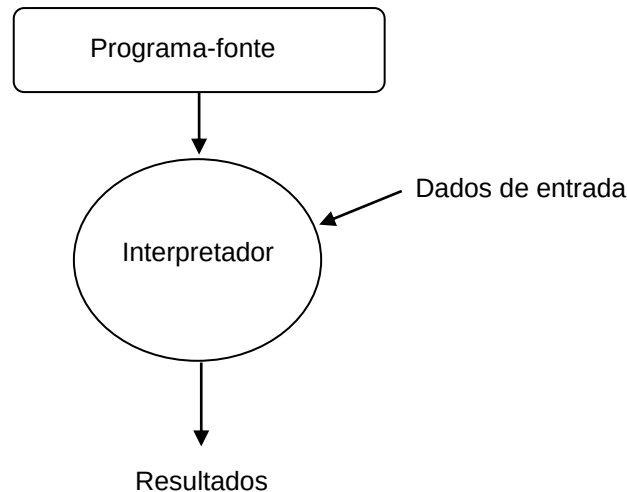


FIGURA 12: INTERPRETAÇÃO PURA.

2.2.3 Sistemas de Interpretação Híbridos

Alguns sistemas de implementação de linguagem são um meio-termo entre os compiladores e os interpretadores puros; eles traduzem programas em linguagem de alto nível para uma linguagem intermediária projetada para permitir fácil interpretação.

Esse método é mais rápido do que a interpretação pura porque as instruções da linguagem fonte são decodificadas somente uma vez. Essas implementações são chamadas de **sistemas de implementação híbridos**.

O processo usado em um sistema de implementação híbrido é mostrado na figura seguinte. Em vez de traduzir código em linguagem intermediária para código de máquina, ele simplesmente interpreta o código intermediário.

O Java é implementado com um sistema híbrido. Ele é parcialmente compilado para detectar erros antes da interpretação e para simplificar o interpretador.

As implementações iniciais de Java eram todas híbridas. Sua forma intermediária, chamada código de bytes, oferece **portabilidade** a qualquer máquina que tenha um interpretador de código de bytes e um sistema *run-time* associado. Juntos, eles são chamados de Java Virtual Machine.

Hoje, existem sistemas que traduzem código intermediário, como o bytecode gerado por aplicações .NET, diretamente para código de máquina, permitindo uma execução mais eficiente. No entanto, aplicações web continuam sendo carregadas do servidor em forma de código intermediário, como ocorre com JavaScript.

Às vezes, um interpretador pode oferecer tanto implementações compiladas como interpretadas para uma linguagem. Nesses casos, o interpretador é usado para desenvolver e para depurar programas. Então, depois de que um estado (relativamente) livre de problemas (*bugs*) é alcançado, os programas são compilados para aumentar sua velocidade de execução.

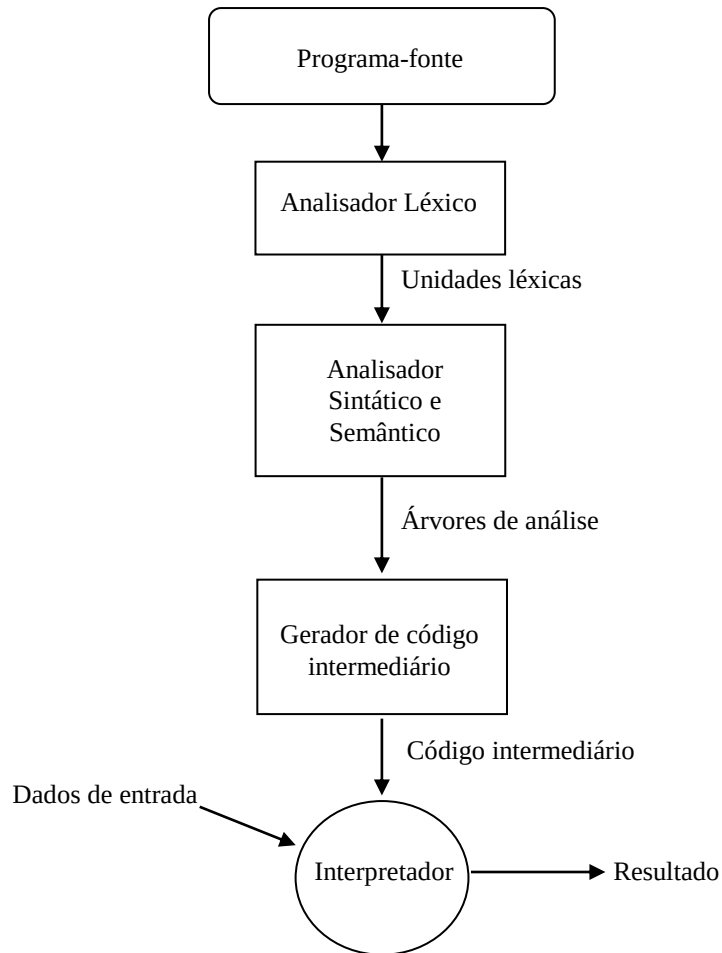


FIGURA 13: SISTEMA DE INTERPRETAÇÃO HÍBRIDO.

🔴* **Exercício:**

Montar uma tabela com as vantagens e desvantagens da Compilação, Interpretação Pura e Híbrida; citando um exemplo de cada.

Compilação	Interpretação Pura	Híbrida

3 O PAPEL DO ANALISADOR SINTÁTICO

O **Analizador Sintático** agrupa os *tokens* fornecidos pelo analisador léxico em estruturas sintáticas, construindo a Árvore Sintática correspondente. Esta Árvore de Derivação, como também é conhecida, pode ser construída através de representações de uma estrutura de dados.

Para isso, utiliza-se uma série de regras de sintaxe, que constituem a gramática da linguagem fonte. É a gramática da linguagem que define a estrutura sintática do programa fonte. Observe abaixo uma possível árvore de derivação gerada por um analisador sintático:

```
<atribuição> ⇒ <id> := <expr>
<id>          ⇒ A | B | C
<expr>        ⇒ <id> + <expr> | <id> * <expr> | (<expr>) | <id>
```

Sentença derivada: **A := B * (A + C)**

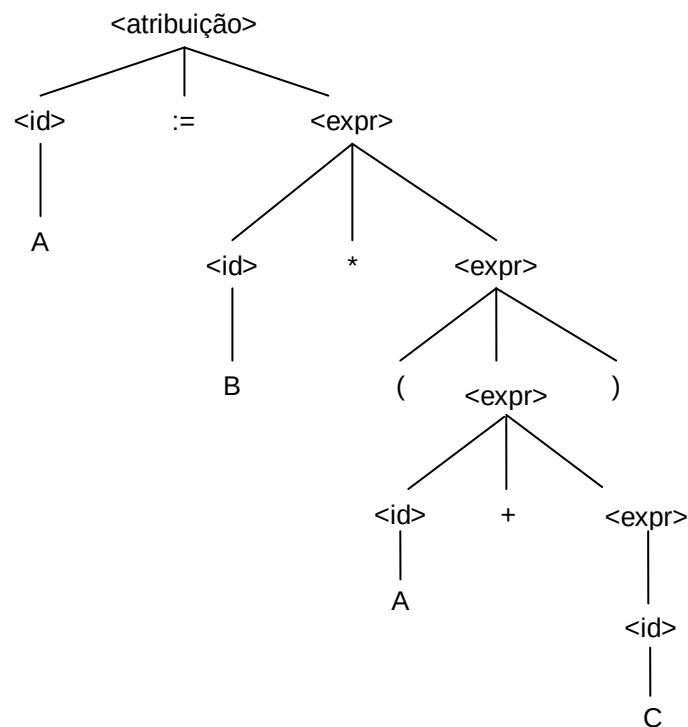


FIGURA 14: ÁRVORE DE DERIVAÇÃO

A Árvore de Análise é a saída lógica do Analisador Sintático. No entanto, o que é realmente feito é guardar as informações relevantes da árvore em uma estrutura do tipo pilha.

O analisador sintático tem também por tarefa o reconhecimento de erros sintáticos, que são construções do programa fonte que não estão de acordo com as regras de formação de estruturas sintáticas como especificado pela gramática.

Em nosso modelo de compilador, o Analisador Sintático obtém uma cadeia de *tokens* proveniente do Analisador Léxico e verifica se a mesma pode ser gerada pela gramática da linguagem-fonte.

Esperamos que o Analisador Sintático relate quaisquer erros de sintaxe de uma forma inteligível. Deve também se recuperar dos erros que ocorram mais comumente, a fim de poder

continuar processando o resto de sua entrada. Os Analisadores Sintáticos devem ser projetados de modo que possam prosseguir na análise, até o fim do programa, mesmo que encontrem erros no texto fonte.

Há duas estratégias básicas para a análise sintática:

1. Estratégia TOP-DOWN ou DESCENDENTE.
2. Estratégia BOTTOM-UP ou REDUTIVA.

No material que segue, vamos analisar os tópicos básicos principais destas duas estratégias.

Os métodos de análise baseados na estratégia top-down constroem a árvore de derivação a partir do símbolo inicial da gramática (raiz da árvore), fazendo a árvore crescer até atingir suas folhas.

A estratégia bottom-up realiza a análise no sentido inverso, isto é, a partir dos tokens do texto fonte (folhas da árvore de derivação) constrói a árvore até o símbolo inicial da gramática.

Na estratégia top-down, em cada passo, um lado esquerdo de produção é substituído por um lado direito (expansão); na estratégia bottom-up, em cada passo, um lado direito de produção é substituído por um símbolo não-terminal (redução).

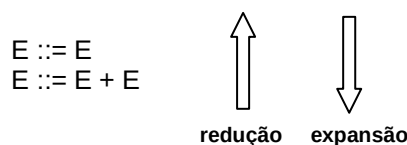


FIGURA 15: REDUÇÃO E EXPANSÃO

Nesta apostila, assume-se que a saída de um analisador sintático seja alguma representação da árvore gramatical para o fluxo de *tokens* produzido pelo analisador léxico.

Na prática, existe certo número de tarefas que poderiam ser conduzidas durante a análise sintática, tais como coletar informações sobre os vários *tokens* na tabela de símbolos, realizar a verificação de tipos e outras formas de análise semântica e até mesmo gerar o código intermediário.

3.1 ANÁLISE SINTÁTICA TOP-DOWN (OU DESCENDENTE)

Esta seção trará a introdução das ideias básicas da análise sintática *top-down*.

3.1.1 Análise Sintática de Descendência Recursiva

A análise sintática *top-down* pode ser vista como uma tentativa de se encontrar uma derivação mais à esquerda para uma cadeia de entrada. Equivalentemente, pode ser vista como uma tentativa de se construir uma árvore gramatical, para a cadeia de entrada, a partir da raiz, criando os nós da árvore gramatical em pré-ordem.

Consideramos agora uma forma geral de análise sintática *top-down*, chamado de descendência recursiva, que pode envolver retrocesso, ou seja, a realização de esquadrinhamentos repetidos da entrada. Este retrocesso é conhecido como *backtracking*.

Por outro lado, os analisadores sintáticos com retrocesso não são vistos muito frequentemente. Uma razão está em que o retrocesso é raramente necessitado para analisar sintaticamente construções de linguagens de programação. Em situações tais como a análise sintática de linguagens naturais, o retrocesso ainda é ineficiente e métodos tabulares são preferidos.

Esta análise faz a expansão da árvore de derivação a partir da raiz, expandindo sempre o não terminal mais à esquerda. Quando existe mais de uma regra de produção para o não-terminal a ser expandido, a opção escolhida é função do símbolo corrente na fita de entrada (token sob o cabeçote de leitura).

Se o token de entrada não define univocamente a produção a ser usada, então todas as alternativas vão ser tentadas até que se obtenha sucesso (ou até que a análise falhe irremediavelmente).

Exemplo: Funcionamento de um analisador descendente com retrocesso.

Ex: Considerando a gramática:

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow ab \mid a \end{aligned}$$

e a cadeia de entrada $w = cad$.

Para construir uma árvore gramatical para esta cadeia, de cima para baixo, criamos inicialmente uma árvore consistindo de um único nó rotulado S . O apontador da fita de entrada aponta para c , o primeiro símbolo de w . Em seguida, usamos a primeira produção para S a fim de expandir a árvore e obter a da figura a seguir, letra (a).

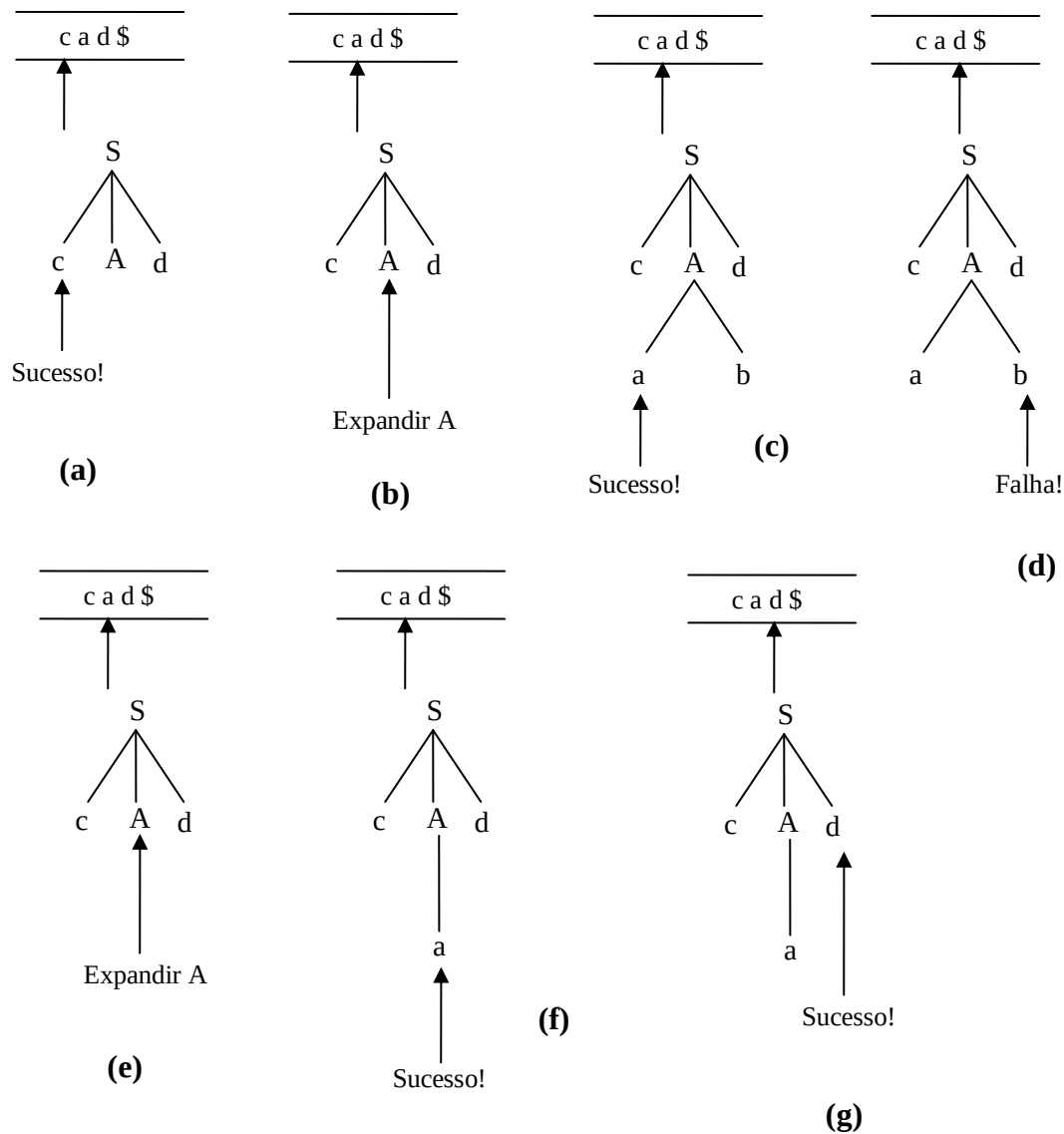


FIGURA 16: ETAPAS NUMA ANÁLISE SINTÁTICA TOP-DOWN.

A folha mais à esquerda, rotulada *c*, reconhece o primeiro símbolo de *w* e, por conseguinte, avançamos o apontador da fita de entrada para *a*, o segundo símbolo de *w*, e consideramos a próxima folha, rotulada *A*. Em seguida, expandimos *A* usando a sua primeira alternativa, obtendo a árvore da figura (c).

Temos agora um reconhecimento para o segundo símbolo da entrada *e*, consequentemente, avançamos o apontador da fita de entrada para *d*, o terceiro símbolo da entrada, e comparamos *d* com a próxima folha, rotulada *b*. Como *b* não é igual a *d*, reportamos uma falha apresentada na figura (d) e retornamos à folha *A* para verificar se existe uma outra alternativa que não tenhamos tentado ainda, mas que poderia produzir um reconhecimento.

Voltando para *A*, precisamos restabelecer o apontador da fita de entrada para a posição 2, aquela que o mesmo detinha quando passamos pela primeira vez por *A*, o que significa que o procedimento para *A* precisa armazenar o apontador da entrada numa variável local.

Tenta-se a segunda alternativa de *A* para obter a árvore na figura (f). A folha *a* reconhece o segundo símbolo de *w* e a folha *d* o terceiro, na figura (g). Uma vez que se produz uma árvore gramatical para *w*, finaliza e anuncia o término com sucesso da análise sintática.

Uma gramática recursiva à esquerda pode levar um analisador sintático de descendência recursiva, mesmo com retrocesso, a um laço infinito. Isto é, quando tentamos expandir A , podemos eventualmente nos encontrar de novo tentando expandir A sem ter consumido nenhum símbolo da entrada. Em virtude disso, este tipo de analisador top-down necessita da eliminação de recursão à esquerda.

$$A ::= Aa \mid b$$

$$A ::= bA'$$

$$A' ::= aA' \mid \varepsilon$$

O processo de retroceder no reconhecimento e tentar produções alternativas dá-se o nome de *backtracking* conforme dito anteriormente. Tal processo é ineficiente, pois leva à repetição da leitura de partes da sentença de entrada e, por isso, em geral, não é usado no reconhecimento de linguagens de programação.

Como o reconhecimento geralmente é acompanhado da execução de ações semânticas (por exemplo, armazenamento de identificadores na Tabela de Símbolos), a ocorrência de retrocesso pode levar o analisador sintático a ter que desfazer essas ações.

Outra desvantagem dessa classe de analisadores é que, quando ocorre um erro, fica difícil indicar com precisão o local do erro, devido à tentativa de aplicação de produções alternativas.

Na seção seguinte, é apresentado um analisador descendente que evita o retrocesso.

3.1.2 Análise Sintática Não-Recursiva Preditiva

É possível implementar analisadores sintáticos sem retrocesso, onde o símbolo sob o cabeçote de leitura define univocamente a produção a ser aplicada.

Esses analisadores são chamados não recursivos preditivos e, para eles, o símbolo sob o cabeçote de leitura determina exatamente qual produção deve ser aplicada na expansão de cada não-terminal. Esses analisadores exigem:

- (1) Que a gramática não tenha recursividade à esquerda,
- (2) Que a gramática esteja fatorada à esquerda e
- (3) Que, para os não-terminais com mais de uma regra de produção, os primeiros terminais deriváveis sejam capazes de identificar, univocamente, a produção que deve ser aplicada a cada instante da análise.

Portanto, para construir-se um analisador preditivo, deve ser possível determinar, dado o símbolo a sob a fita de leitura e o não-terminal A a ser derivado, qual das produções alternativas $A \rightarrow \beta_1 \mid \beta_2 \mid \beta_3 \mid \dots \mid \beta_n$ é a que deriva, unicamente, a sequência que inicia por a .

Para isso, é necessário fatorar a gramática:

$$S ::= aBc \mid bA$$

$$A ::= cB \mid baS \mid \varepsilon$$

$$B ::= caA \mid cbS \mid \varepsilon$$

$$B ::= cB' \mid \varepsilon$$

$$B' ::= aA \mid bS$$

3.1.3 Análise Preditiva Tabular (não recursiva)

É possível construir analisadores preditivos *não recursivos* que implementam um autômato de pilha controlado por uma tabela de análise.

O princípio do reconhecimento preditivo é a determinação da produção a ser aplicada, cujo lado direito irá substituir o símbolo não-terminal que se encontra no topo da pilha. O analisador busca a produção a ser aplicada na tabela de análise, levando em conta o não-terminal no topo da pilha e o *token* sob o cabeçote de leitura.

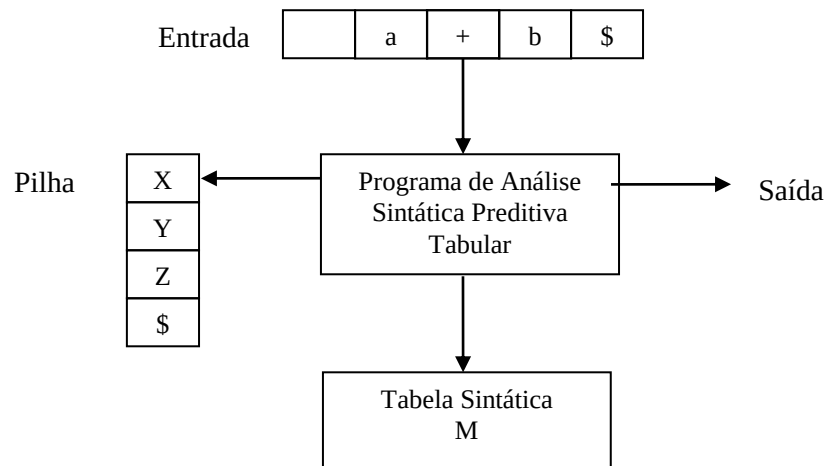


FIGURA 17: ANALISADOR PREDITIVO TABULAR.

Um analisador preditivo orientado por tabela compreende uma fita de entrada, uma pilha e uma tabela de análise, conforme a figura acima. A fita de entrada contém a sentença a ser analisada seguida de \$, símbolo que marca o fim da sentença.

Inicialmente, a pilha contém \$, que marca a sua base, seguido do símbolo inicial da gramática. A tabela de análise é uma matriz **M** com **n** linhas e **t+1** colunas, onde **n** é o número de símbolos não-terminais, e **t** é o número de símbolos terminais (a coluna extra corresponde ao símbolo \$).

O analisador é controlado por um programa que se comporta conforme descrito a seguir. Considerando **X** como o símbolo do topo da pilha e **a** como terminal da fita de entrada sob o cabeçote de leitura, o analisador executa uma de três ações possíveis:

1. Se **X = a = \$**, ou seja, **se topo da pilha e símbolo do buffer for \$**, o analisador pára, aceitando a sentença;
2. Se **X = a <> \$**, ou seja, **se o topo da pilha e símbolo do buffer forem iguais, mas forem diferentes de \$**, o analisador desempilha **a** e avança o cabeçote de leitura para o próximo símbolo na fita de entrada;
3. Se **X** é um símbolo não-terminal, o analisador consulta a entrada **M[X, a]** da tabela de análise. Essa entrada poderá conter uma produção da gramática ou ser vazia. Supondo **M[X, a] = {X → UVW}**, o analisador substitui **X** (que está no topo da pilha) por **WVU** (ficando **U** no topo) e retorna a produção aplicada. Se **M[X, a]** é vazia, isso corresponde a uma situação de erro; neste caso, o analisador chama uma rotina de tratamento de erro.

O comportamento do analisador pode ser descrito através de uma tabela que mostra, a cada passo, o conteúdo da pilha e o restante da sentença a ser lida, conforme é exemplificado a seguir.

Exemplo: Movimentos de um analisador tabular preditivo.

Considere a gramática não-ambígua abaixo:

$$\begin{aligned} E &\rightarrow EvT \mid T \\ T &\rightarrow TeF \mid F \\ F &\rightarrow cF \mid id \end{aligned}$$

Eliminando-se a recursividade à esquerda das produções que definem E e T, obtém-se:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow vTE' \mid \varepsilon \\ T &\rightarrow FT' \\ T' &\rightarrow eFT' \mid \varepsilon \\ F &\rightarrow cF \mid id \end{aligned}$$

A tabela de análise preditiva para essa gramática é mostrada a seguir:

	Id	v	e	c	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	
E'		$E' \rightarrow vTE'$			$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$	
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow eFT'$		$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow cF$	

TABELA de análise para a GLC acima.

Com a sentença **idvideid**, o reconhecedor preditivo realiza os movimentos mostrados na tabela que segue. Inicialmente, o cabeçote aponta para o símbolo mais à esquerda da sentença de entrada. Observando as ações produzidas pelo reconhecedor, pode-se notar que as produções usadas na análise constituem uma derivação mais à esquerda da sentença.

Pilha	Entrada	Ação
\$E	id v id e id \$	$E \rightarrow TE'$
\$E'T	id v id e id \$	$T \rightarrow FT'$
\$E'T'F	id v id e id \$	$F \rightarrow id$
\$E'T'id	id v id e id \$	Desempilha e lê símbolo
\$E'T'	v id e id \$	$T' \rightarrow \epsilon$
\$E'	v id e id \$	$E' \rightarrow vTE'$
\$E'Tv	v id e id \$	Desempilha e lê símbolo
\$E'T	id e id \$	$T \rightarrow FT'$
\$E'T'F	id e id \$	$F \rightarrow id$
\$E'T'id	id e id \$	Desempilha e lê símbolo
\$E'T'	e id \$	$T' \rightarrow eFT'$
\$E'T'Fe	e id \$	Desempilha e lê símbolo
\$E'T'F	id \$	$F \rightarrow id$
\$E'T'id	Id \$	Desempilha e lê símbolo
\$E'T'	\$	$T' \rightarrow \epsilon$
\$E'	\$	$E' \rightarrow \epsilon$
\$	\$	Aceita a sentença em 17 passos!

↓
Derivações da sentença!

TABELA – Movimentos de um analisador tabular preditivo.

Na implementação de um analisador preditivo tabular, a maior dificuldade está na construção da tabela de análise. Para construir essa tabela, é necessário computar duas funções associadas às gramáticas: as funções FIRST e FOLLOW.

Possíveis erros na pilha:

- 1) Não existe mapeamento na tabela para topo da pilha e símbolo da entrada.
- 2) O estado final da pilha e da entrada não é \$ (vazio).
- 3) Símbolo da entrada não existe no alfabeto da linguagem expressa na tabela.

Exercício 1: Verificar se as sentenças abaixo são reconhecidas como válidas para a gramática do exemplo anterior.

- a) ideid
- b) ccideid
- c) ideidc
- d) idecidid
- e) idvid
- f) cidvvid
- g) idveid
- h) cideidvid

Exercício 2: Usando a tabela de análise abaixo, diga se a sentença proposta é reconhecida ou não, e o número de passos necessária na pilha.

GLC:

$S ::= aCb \mid bA \mid cBc$
 $A ::= aB \mid \varepsilon$
 $B ::= aAc \mid bCb$
 $C ::= aA \mid cAb$

Tabela:

	a	b	c	\$
S	S $\rightarrow$ aCb	S $\rightarrow$ bA	S $\rightarrow$ cBc	-
A	A $\rightarrow$ aB	A $\rightarrow \varepsilon$	A $\rightarrow \varepsilon$	A $\rightarrow \varepsilon$
B	B $\rightarrow$ aAc	B $\rightarrow$ bCb	-	-
C	C $\rightarrow$ aA	-	C $\rightarrow$ cAb	-

- a) acbb
- b) cbabc
- c) acaacbb
- d) baacc

Exercício 3: Usando a tabela de análise abaixo, diga se a sentença proposta é reconhecida ou não, e o número de passos necessária na pilha.

GLC:

$S ::= aBc \mid cCb$
 $A ::= bCc \mid cB$
 $B ::= baA \mid \varepsilon$
 $C ::= aAc \mid caS$

Tabela:

	a	b	c	\$
S	S $\rightarrow$ aBc	-	S $\rightarrow$ cCb	-
A	-	A $\rightarrow$ bCc	A $\rightarrow$ cB	-
B	-	B $\rightarrow$ baA	B $\rightarrow \varepsilon$	-
C	C $\rightarrow$ aAc	-	C $\rightarrow$ caS	-

- a) abacc
- b) abacbacc
- c) abacbac
- d) ccacacbb

3.1.4 FIRST

Se α é uma forma sentencial (sequência de símbolos da gramática), então $\text{FIRST}(\alpha)$ é o conjunto de terminais que iniciam formas sentenciais derivadas a partir de α . Se $\alpha \rightarrow^* \varepsilon$, então a palavra vazia também faz parte do conjunto.

Para calcular $\text{FIRST}(X)$ para um símbolo X da gramática, aplicam-se as regras abaixo, até que não se possa adicionar mais terminais ou ε ao conjunto em questão.

- 1) O FIRST de um não terminal que está à esquerda de $::=$ é sempre o primeiro(first) símbolo terminal das suas produções.
- 2) Se $X \rightarrow \varepsilon$, adicionar ε a $\text{FIRST}(X)$.
- 3) Se $x \rightarrow Y_1, Y_2, \dots, Y_n$ é uma produção e, alguma destas produções deriva ε , então $\text{FIRST}(Y_i)$ está em $\text{FIRST}(X)$, ou seja, todos os $\text{FIRST}(Y_n)$ estão em $\text{FIRST}(X)$. Se todo Y_i deriva ε , então ε está em $\text{FIRST}(X)$.
- 4) Se o primeiro símbolo destas produções for um não terminal, copia-se o conjunto FIRST deste símbolo para o conjunto FIRST do símbolo ao qual se está analisando, menos o ε .

Exemplo:

$S ::= Ab \mid a$

$A ::= Bd \mid \varepsilon$

$B ::= c$

$\text{FIRST}(S) = a, b, c$

$\text{FIRST}(A) = \varepsilon, c$

$\text{FIRST}(B) = c$

Algoritmo:

(para todo $X \in V_n \cup V_t$)

1 – Se $X \in t \rightarrow \text{first}(X) = \{X\}$

2 – Se $X \in N \wedge X \rightarrow a\alpha \in P \rightarrow a \in \text{First}(X)$

Obs: Se $X \rightarrow \varepsilon \in P \rightarrow \varepsilon \in \text{First}(X)$

3 – Se $X \rightarrow y_1 y_2 \dots y_k \in P \rightarrow \text{First}(y_1) \in \text{First}(X)$

Se $\varepsilon \in \text{First}(y_1) \rightarrow \text{First}(y_2)$ também $\in \text{first}(X)$

Se $\varepsilon \in \text{First}(y_2) \rightarrow \dots$

Se $\varepsilon \in \text{First}(y_k) \rightarrow \varepsilon$ também $\in \text{First}(X)$

Veja:

Se:

$X ::= aB$

Então:

$\text{FIRST}(X) = a$

Se:

$X ::= AbB$

$A ::= aA \mid \varepsilon$

$B ::= aB \mid B$

Então:

$\text{FIRST}(X) = a, b$

Se:

$X ::= ABc$

$A ::= aA \mid \varepsilon$

$B ::= bB \mid \varepsilon$

Então:

$\text{FIRST}(X) = a, b, c$

Se:

$X ::= AB$

$A ::= aA \mid \varepsilon$

$B ::= bB \mid \varepsilon$

Então:

$\text{FIRST}(X) = a, b, \varepsilon$

3.1.4.1 FIRST - Exercícios propostos

Encontre o conjunto FIRST dos símbolos das gramáticas abaixo:

1-

$S ::= Ab \mid Bbc$

$A ::= aA \mid \varepsilon$

$B ::= bB \mid Ad \mid \varepsilon$

2-

$S ::= ABC$

$A ::= aA \mid \varepsilon$

$B ::= bB \mid ACd$

$C ::= cC \mid \varepsilon$

3-

$S ::= Bz \mid CAw \mid AB$

$A ::= aCB \mid Byb \mid bC$

$B ::= cAd \mid Byd \mid aB \mid \varepsilon$

$C ::= zBd \mid wCc \mid ABY \mid \varepsilon$

4-

$S ::= xSy \mid ABz \mid ACw \mid BA$

$A ::= aCB \mid Bby \mid By$

$B ::= cCy \mid Cdy \mid \varepsilon$

$C ::= dBz \mid cCw \mid ABY \mid \varepsilon$

5-

$S ::= xSy \mid ABz \mid BA$
 $A ::= aCB \mid By \mid \epsilon$
 $B ::= cCy \mid Cdy \mid \epsilon$
 $C ::= dBz \mid By$

6-

$A ::= bBc \mid Bcd \mid cdB \mid c$
 $B ::= Ac \mid dAB \mid \epsilon$
 $C ::= cBc \mid Aa$

7-

$S ::= aAb \mid Bc \mid BcAb$
 $A ::= Sac \mid aAc \mid ab$
 $B ::= Ac \mid aBb \mid ab \mid \epsilon$

8-

$A ::= cB \mid bC \mid Cd$
 $B ::= ac \mid Cd \mid aA$
 $C ::= Bb \mid caA \mid \epsilon$

9-

$S ::= a \mid BA \mid Aac$
 $A ::= Bab \mid ab \mid bS$
 $B ::= aS \mid aAc \mid \epsilon$

10-

$S ::= Aa \mid bBa \mid aaC$
 $A ::= ab \mid aBC \mid bSc \mid \epsilon$
 $B ::= Aca \mid bbA \mid \epsilon$
 $C ::= bS \mid Abc \mid AB$

3.1.5 FOLLOW

A função FOLLOW é definida para símbolos não-terminais. Sendo A um não-terminal, FOLLOW(A) é o conjunto de terminais **b** que podem aparecer imediatamente à direita de A em alguma forma sentencial. Isto é, o conjunto de terminais **b**, tal que existe uma derivação da forma $S \rightarrow^* \alpha Ab\beta$ para α e β quaisquer.

Para calcular FOLLOW sigamos os seguintes passos:

- 1) Se S é o símbolo inicial da gramática e \$ é o marcador de fim da sentença, então \$ está em FOLLOW(S).
- 2) Esqueça os símbolos à esquerda de $::=$. O FOLLOW de todos os NT da produção é sempre o FIRST do que vem imediatamente depois dele. Se for terminal o que vier depois, copia este terminal para a tabela FOLLOW, se for NT copia o conjunto FIRST dele (com exceção do epsilon).
- 3) Analise a gramática da direita para a esquerda. Se a produção inicia com NT, por exemplo $A ::= B$, então, adicionar ao FOLLOW(B) o FOLLOW(A). Caso o FIRST(B) possuir ϵ , fazer a mesma coisa com o próximo símbolo da produção. Exemplo: $A ::= BC$. Caso C possua ϵ , fazer o terceiro passo com o B também.

Algoritmo:

(Para todo $A \in NT$)

1 – Se A é o símbolo inicial da gramática

→ $\$ \in \text{Follow}(A)$

2 – Se $A \rightarrow \alpha B \beta \in P \wedge \beta \neq \varepsilon$

→ adicione $\text{first}(\beta)$ em $\text{Follow}(B)$

3 – Se $A \rightarrow \alpha B$ (ou $A \rightarrow \alpha B \beta$, onde $\varepsilon \in \text{First}(\beta)$) $\in P$

→ adicione $\text{Follow}(A)$ em $\text{Follow}(B)$

Veja:

em $A ::= aBb \Rightarrow \text{Follow}(B) = b$

em $A ::= aBC \Rightarrow \text{Follow}(B) = \text{First}(C) - \varepsilon$

em $A ::= aB \Rightarrow \text{Follow}(B) = \text{Follow}(A)$ e se $A ::= aBC$ e $\varepsilon \in \text{First}(C)$ então:

$\text{Follow}(B) = \text{Follow}(A)$

Exemplo:

$S ::= aAb \mid Bc \mid \varepsilon$

$A ::= aBb \mid cS$

$B ::= dAC \mid a$

$C ::= aB \mid \varepsilon$

FIRST:

$S = \{a, \varepsilon, d\}$

$A = \{a, c\}$

$B = \{d, a\}$

$C = \{a, \varepsilon\}$

FOLLOW:

$S = \{\$, b, a, c\}$

$A = \{b, a, c\}$

$B = \{c, b\}$

$C = \{c, b\}$

3.1.5.1 FOLLOW - Exercícios Propostos

Encontre o conjunto FOLLOW (e consequentemente o FIRST) das questões abaixo:

1-

$S ::= ABC$
 $A ::= aA \mid \epsilon$
 $B ::= bB \mid ACd$
 $C ::= cC \mid \epsilon$

2-

$S ::= xSy \mid ABz \mid ACw \mid BA$
 $A ::= aCB \mid Bby \mid By$
 $B ::= cCy \mid Cdy \mid \epsilon$
 $C ::= dBz \mid cCw \mid ABY \mid \epsilon$

3-

$S ::= AbBC \mid DE$
 $A ::= aA \mid \epsilon$
 $B ::= DBE \mid c$
 $C ::= BC \mid dCd \mid \epsilon$
 $D ::= eD \mid \epsilon$
 $E ::= ES \mid fE \mid g$

4-

$S ::= AC \mid CeB \mid Ba$
 $A ::= aA \mid BC$
 $B ::= bB \mid AB \mid \epsilon$
 $C ::= cC \mid \epsilon$

5-

$S ::= AB \mid Abc$
 $A ::= aA \mid \epsilon$
 $B ::= bB \mid ABd \mid \epsilon$

6-

$S ::= AaB$
 $A ::= aAS \mid \epsilon$
 $B ::= b \mid ACd$
 $C ::= cC \mid \epsilon$

7-

$S ::= Aab \mid BcA \mid ScAb$
 $A ::= SacB \mid BSa \mid ab \mid \epsilon$
 $B ::= Ac \mid aBbS \mid ab$

8-

$S ::= Aa \mid Bb \mid SAc$
 $A ::= Sab \mid ab \mid \epsilon$
 $B ::= BAa \mid BA \mid Ae \mid \epsilon$

9-

$S ::= AaBc \mid ABb \mid abC$
 $A ::= aaS \mid aBC \mid SbB$
 $B ::= BAc \mid acA \mid \epsilon$
 $C ::= bb \mid bcS$

3.1.6 Construção da Tabela de Parsing (Tabela de Análise)

Entrada: Gramática G

Saída: Tabela de Análise M

Método:

- 1) Para cada produção $A \rightarrow \alpha$ de G, execute os passos 2 e 3 (para criar a linha A da tabela M).
- 2) Para cada terminal **a** de **FIRST(α)**, adicione a produção $A \rightarrow \alpha$ a **M[A,a]**.
- 3) Se **FIRST(α)** inclui a palavra vazia, então adicione $A \rightarrow \alpha$ a **M[A,b]** para cada **b** em **FOLLOW(A)**.

Aplicando o algoritmo acima à gramática G sugerida, obtém-se as seguintes entradas para a tabela:

G =

$E \rightarrow TE'$
 $E' \rightarrow vTE' \mid \epsilon$
 $T \rightarrow FT'$
 $T' \rightarrow eFT' \mid \epsilon$
 $F \rightarrow cF \mid id$

Para $E \rightarrow TE'$ tem-se $FIRST(TE') = \{c, id\}$ $M[E, c] = M[E, id] = E \rightarrow TE'$
 Para $E' \rightarrow vTE'$ tem-se $FIRST(vTE') = \{v\}$ $M[E', v] = E' \rightarrow vTE'$
 Para $E' \rightarrow \epsilon$ tem-se $FOLLOW(E') = \{\$ \}$ $M[E', \$] = E' \rightarrow \epsilon$
 Para $T \rightarrow FT'$ tem-se $FIRST(FT') = \{c, id\}$ $M[T, c] = M[T, id] = T \rightarrow FT'$
 Para $T' \rightarrow eFT'$ tem-se $FIRST(eFT') = \{e\}$ $M[T', e] = T' \rightarrow eFT'$
 Para $T' \rightarrow \epsilon$ tem-se $FOLLOW(T') = \{v, \$ \}$ $M[T', v] = M[T', \$] = T' \rightarrow \epsilon$
 Para $F \rightarrow cF$ tem-se $FIRST(cF) = \{c\}$ $M[F, c] = F \rightarrow cF$
 Para $F \rightarrow id$ tem-se $FIRST(id) = \{id\}$ $M[F, id] = F \rightarrow id$

	id	v	e	c	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	
E'		$E' \rightarrow vTE'$			$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$	
T'		$T' \rightarrow \epsilon$	$T' \rightarrow eFT'$		$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow cF$	

TABELA de Parsing para a gramática proposta

Se, em cada entrada da Tabela de Análise, existe apenas uma produção, então a gramática que originou a tabela é dita do **tipo LL(1)**.

Isso significa que as sentenças geradas pela gramática são passíveis de serem analisadas da esquerda para a direita (**Left to right**), produzindo uma derivação mais à esquerda (**Leftmost derivation**), levando em conta apenas **(1)** símbolo da entrada.

O método apresentado para gerar a tabela pode ser aplicado a qualquer gramática. Contudo, para certas gramáticas, a tabela resultante poderá ter entradas multiplamente definidas. Isso ocorre para as gramáticas ambíguas. Consequentemente, essas gramáticas não são do tipo LL(1).

No entanto, as gramáticas devem atender as seguintes condições:

- 1) Não possuir recursão à esquerda.
- 2) Estar fatorada.
- 3) Para os NT com mais de uma regra de produção, os primeiros terminais deriváveis sejam capazes de identificar, univocamente, a produção que deve ser aplicada a cada instante da análise.

Somente GLC's que satisfazem estas condições podem ser analisadas (deterministicamente) pelos Analisadores Descendentes LL(1).

3.1.6.1 Tabela de Parsing - Exercícios Propostos

Monte a tabela de análise para as gramáticas e após verifique se as sentenças propostas são reconhecidas.

a)

$S ::= aBb \mid bAc \mid cCb$
 $A ::= aCb \mid \epsilon$
 $B ::= aCa \mid bAb$
 $C ::= aB \mid cAc$

Sentenças: 1→ aaccab 2→ baabbbc 3→ babbca 4→ aacacbcab

b)

$S ::= aBa \mid AB \mid dA$
 $A ::= bA \mid c$
 $B ::= bA \mid cC \mid dAa \mid \epsilon$
 $C ::= aCA \mid b$

Sentenças: 1→ adcaa 2→ dbbbbc 3→ ccabbc 4→ acbbca

c)

$S ::= aA$
 $A ::= aBc \mid \epsilon$
 $B ::= cCa$
 $C ::= bB \mid c$

Sentenças: 1→ aaccac 2→ aacbcaa 3→ aacbccaac 4→ aacbbccaac

d)

$S ::= cBb \mid bCa \mid abC$
 $A ::= aCb \mid cBC \mid bB$
 $B ::= acA \mid \epsilon$
 $C ::= cb \mid bcS$

Sentenças: 1→ cacbcb 2→ bbcabcba 3→ abbccacbb 4→ caccacbcbb

e)

$S ::= aAd \mid B$

$A ::= bB \mid \varepsilon$

$B ::= dBa \mid cCb$

$C ::= aSb \mid \varepsilon$

Sentenças: $1 \rightarrow abcaabcbddbbd$ $2 \rightarrow abddcabbaad$ $3 \rightarrow abddcaabddcaad$ $4 \rightarrow abddcacbbbaad$

f) Criar uma GLC (LL1) com no mínimo quatro regras, sendo três delas com duas produções e no mínimo uma produção de ε . Fazer a tabela e criar quatro exercícios nos moldes dos anteriores.

3.2 ANÁLISE SINTÁTICA BOTTOM-UP

O estilo geral de análise sintática *bottom-up* é conhecido como análise de empilhar e reduzir, também conhecido como análise LR.

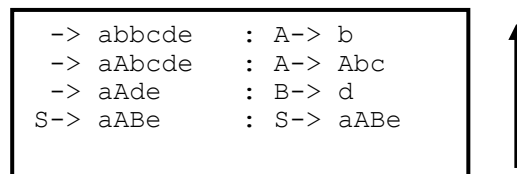
A análise gramatical de empilhar e reduzir tenta construir uma árvore gramatical para uma cadeia de entrada começando pelas folhas (o fundo) e trabalhando árvore acima em direção à raiz (o topo). Podemos pensar neste processo como o de “reduzir” uma cadeia w ao símbolo de partida de uma gramática.

A cada passo de *redução*, uma subcadeia particular, que reconheça o lado direito de uma produção, é substituída pelo símbolo à esquerda daquela produção e, se a subcadeia tiver sido escolhida corretamente a cada passo, uma derivação mais à direita terá sido rastreada na ordem inversa.

Ex: Considere a gramática [AHO].

$S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$

A sentença *abbcde* pode ser reduzida a S pelos seguintes passos:

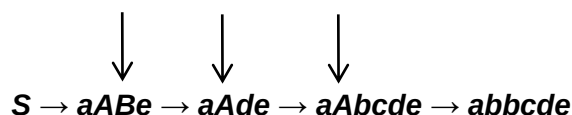


Pode-se esquadrihar *abbcde* procurando por uma subcadeia que reconheça o lado direito de alguma produção. As subcadeias b e d se qualificam. Vamos escolher o b mais à esquerda e substituí-lo por A , o lado esquerdo da produção $A \rightarrow b$; obtemos dessa forma a cadeia *aAbcde*. Agora as subcadeias *Abc*, b e d reconhecem o lado direito de alguma produção.

Apesar de b ser a subcadeia mais à esquerda que reconheça o lado direito de alguma produção, escolhemos substituir a subcadeia *Abc* por A , o lado esquerdo da produção $A \rightarrow Abc$. Obtemos agora *aAde*. Com a substituição de d por B , o lado esquerdo da produção $B \rightarrow d$, obtemos *aABe*.

Podemos agora substituir toda esta cadeia por S . Consequentemente, através de uma sequência de quatro reduções, estamos capacitados a reduzir *abbcde* a S .

Essas reduções, de fato, rastreiam a seguinte derivação mais à direita, na ordem reversa:



Observação: Processado na ordem reversa em relação ao não-terminal mais à esquerda.

3.2.1 Análise Sintática de Precedência de Operadores

Para uma pequena, porém importante classe de gramáticas, chamadas gramáticas de precedência de operadores, podemos facilmente construir manualmente eficientes analisadores sintáticos de empilhar e reduzir (shift-reduce).

Essas gramáticas possuem a propriedade (dentre outras exigências essenciais) de que nenhum lado direito de produção seja ϵ , ou tenha dois não-terminais adjacentes. Uma gramática com a última propriedade é chamada de *gramática de operadores*.

Ex: A seguinte gramática para expressões [AHO],

$$\begin{aligned} E &\rightarrow EAE \mid (E) \mid -E \mid \text{id} \\ A &\rightarrow + \mid - \mid * \mid / \end{aligned}$$

não é uma gramática de operadores porque o lado direito EAE possui dois (de fato três) não-terminais consecutivos. Entretanto, se substituirmos A por cada uma de suas alternativas, obtemos a seguinte gramática de operadores:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid -E \mid \text{id}$$

Descreve-se agora uma técnica de análise sintática fácil de implementar, chamada de Análise Sintática de Precedência de Operadores.

Na Análise Sintática de Precedência de Operadores, definimos três *relações de precedência* disjuntas, $<$, $=$ e $>$, entre certos pares de terminais. Essas relações de precedência guiam a seleção de *handles* (produções da gramática) e têm os seguintes significados:

RELAÇÃO	SIGNIFICADO
$A < B$	A “confere precedência” a B
$A = B$	A “possui a mesma precedência que” B
$A > B$	A “tem precedência sobre” B

Deve-se prevenir o leitor de que, apesar dessas relações parecerem similares às relações aritméticas “menor do que”, “igual a” e “maior do que”, **as relações de precedência possuem propriedades diferentes**.

Por exemplo, poderíamos ter $a < b$ e $a > b$ para a mesma linguagem, ou não termos nenhuma das relações vigorando para o mesmo par de terminais a e b .

O método para construção para a **Tabela de Precedência de Operadores** utilizado nesta apostila, é intuitivo, e está baseado nas noções tradicionais de associatividade e precedência de operadores.

3.2.1.1 Construção da Tabela de Precedência de Operadores

A tabela de precedência é construída comparando os operadores dois a dois, com base em:

- Níveis de precedência dos operadores (exemplo: * e / > + e -)
- Associatividade (geralmente à esquerda para todos)
- Comportamento especial de parênteses e id

Relações de Precedência: a tabela é montada comparando todos os pares de terminais válidos, usando as regras:

- Se o terminal na pilha tem menor precedência que o próximo símbolo: < (Shift)
- Se o terminal na pilha tem maior precedência: > (Reduce)
- Se formam uma estrutura (parênteses): = (Shift)
- Se não há relação válida, é erro: E1 a E4
- \$: representa o fim da entrada e tem menor precedência que todos os outros símbolos

E1 a E4 representam erros sintáticos, como:

- **E1:** falta de operando
- **E2:** parênteses não balanceados
- **E3:** falta de operador
- **E4:** falta de parêntese à direita

3.2.1.2 Analisador Shift-Reduce para Gramáticas de Precedência de Operadores

Considerando a gramática G:

$E ::= E + E$
 $E ::= E - E$
 $E ::= E * E$
 $E ::= E / E$
 $E ::= (E)$
 $E ::= id$

Na gramática acima, a precedência dos operadores é dada pela seguinte tabela:

	Id	+	-	/	*	(	)	\$
Id	E3	>	>	>	>	E3	>	>
+	<	>	>	<	<	<	>	>
-	<	>	>	<	<	<	>	>
/	<	>	>	>	>	<	>	>
*	<	>	>	>	>	<	>	>
(	<	<	<	<	<	<	=	E4
)	E3	>	>	>	>	E3	>	>
\$	<	<	<	<	<	<	E2	E1

TABELA de Precedência de Operadores

O Algoritmo de análise funciona da seguinte forma:

- O reconhecimento é feito da esquerda para a direita;
- Quando a relação de precedência entre o símbolo do topo da pilha e a posição atual da sentença for < ou = é feito um *shift* (empilha símbolo corrente do *buffer* na pilha);
- Quando a relação de precedência entre o símbolo do topo da pilha e a posição atual da sentença for > é feito um *reduce* na pilha. Nesse momento deve ser feita uma verificação dos símbolos que circundam o *handle* para verificar se há erros.

Nos casos em que nenhuma relação de precedência é válida, têm-se situações de erro e1: "falta operando", e2: "parênteses não balanceados", e3: "falta operador" e e4: "falta parênteses da direita". Nesses casos, uma rotina deveria ser chamada para emitir uma mensagem de erro e agir na pilha e no buffer para que a análise possa continuar.

Existem duas situações de detecção de erros, que são:

1. Se não existe relação de precedência determinada na tabela entre o terminal da pilha e o terminal corrente no buffer. No exemplo anterior, e1, e2, e3 e e4.
2. Numa situação de *reduce*, se não existe lado direito nas produções igual ao *handle* (a produção de G) que está na pilha.

Para o exemplo dado anteriormente, teríamos basicamente as seguintes situações de erros em *reduce* possíveis:

- Se +, -, * ou / são reduzidos, verifica-se se aparecem terminais em ambos os lados. Se não, diagnostica-se um erro do tipo “falta um operando”;
- Se id é reduzido, verifica-se se aparece um não terminal em algum dos lados. Se aparecer diagnostica-se um erro do tipo “falta operador”;
- Se () é reduzido, verifica-se se aparece um não-terminal entre os parênteses. Se não houver, diagnostica-se um erro do tipo “nenhuma expressão entre os parênteses”. Ainda nesse caso, verifica-se se aparece um não-terminal do lado externo aos parênteses. Se aparecer, diagnostica-se um erro do tipo “falta operador”.

Exemplo:

Gramática:

$E \Rightarrow E + E$
 $E \Rightarrow E - E$
 $E \Rightarrow E * E$
 $E \Rightarrow E / E$
 $E \Rightarrow (E)$
 $E \Rightarrow id$

Entrada: $id * (id / id)$

3.2.1.3 Exercícios: Autômatos de Pilha Bottom-UP - Shift-Reduce

Gramática de precedência de Operadores:

$E \Rightarrow E + E$
 $E \Rightarrow E - E$
 $E \Rightarrow E * E$
 $E \Rightarrow E / E$
 $E \Rightarrow (E)$
 $E \Rightarrow id$

Verifique na pilha as entradas propostas:

- $id * id + id * (id + id)$
- $(id + id) * (id - id)$
- $(id - id * (id + id)) / id$
- $id + id / id * id$
- $id + id * (id/id)$
- $id / (id - id) * id$
- $id *) id + id$
- $id + id * id + (id * id)$
- $id * (+id)$

3.3 TRATAMENTO DOS ERROS DE SINTAXE

Nesta seção, considera-se a natureza dos erros sintáticos e as estratégias gerais para a sua recuperação. Duas dessas estratégias, chamadas “modalidades do desespero” e recuperação em nível de frase, são discutidas mais pormenorizadamente junto com os métodos individuais de análise sintática. A implementação de cada estratégia requer o julgamento do produtor do compilador, mas daremos algumas diretrizes gerais relacionadas a essas abordagens.

Se um compilador tivesse que processar somente programas corretos, seu projeto e sua implementação seriam grandemente simplificados. Mas os programadores frequentemente escrevem programas incorretos, e um bom compilador deveria assistir o programador na identificação e localização dos erros.

É gritante que, apesar dos erros serem normalmente os mesmos, poucas linguagens sejam projetadas tendo-se o tratamento de erros em mente. Nossa civilização seria radicalmente diferente se as linguagens faladas tivessem as mesmas exigências de correção sintática que as das linguagens de computadores.

A maioria das especificações das linguagens de programação não descreve como um compilador deveria responder aos erros; tal tarefa é deixada ao projetista do compilador. O planejamento do tratamento de erros exatamente desde o início poderia tanto simplificar a estrutura de um compilador quanto melhorar sua resposta aos erros.

Segundo [AHO], os programas podem conter erros em muitos níveis diferentes. Por exemplo, os erros podem ser:

- a) Léxicos, tais como errar a grafia de um identificador, palavra-chave ou operador;
- b) Sintáticos, tais como uma expressão aritmética com parênteses não-balanceados;
- c) Semânticos, tais como um operador aplicado a um operando incompatível;
- d) Lógicos, tais como uma chamada infinitamente recursiva.

Frequentemente, boa parte da detecção e recuperação de erros num compilador gira em torno da fase de análise sintática. Isto porque os erros ou são sintáticos por natureza ou são expostos quando o fluxo de *tokens* proveniente do analisador léxico desobedece às regras gramaticais que definem a linguagem de programação. Outra razão está na precisão dos modernos métodos de análise sintática; que podem detectar muito eficientemente a presença de erros sintáticos num programa.

Detectar precisamente a presença de erros semânticos ou lógicos em tempo de compilação é uma tarefa muito mais difícil. Nesta seção, umas poucas técnicas básicas para a recuperação de erros sintáticos e suas implementações são discutidas em conjunto com os métodos de análise sintática.

O tratador de erros num analisador sintático possui metas simples de serem estabelecidas:

- a) Deve relatar a presença de erros clara e acuradamente.
- b) Deve recuperar-se de cada erro, suficientemente rápido a fim de ser capaz de detectar erros subsequentes.
- c) Não deve retardar significativamente o processamento de programas corretos.

A realização efetiva dessas metas apresenta desafios difíceis.

Felizmente, os erros comuns são simples e frequentemente basta um mecanismo de

tratamento de erros relativamente direto. Em alguns casos, entretanto, um erro pode ter ocorrido muito antes de sua presença ter sido detectada e sua natureza precisa pode ser muito difícil de ser deduzida. Em casos difíceis, o tratador de erros pode ter que adivinhar o que o programador tinha em mente quando o programa foi escrito.

Vários métodos de análise sintática, detectam os erros tão cedo quanto possíveis. Mais precisamente, possuem a *propriedade do prefixo viável*, significando que detectam que um erro ocorreu tão logo tenham examinado um prefixo da entrada que não seja o de qualquer cadeia da linguagem.

Para alguns erros concretos, considerando o seguinte programa Pascal:

```
(1)    program prmax (input, output);
(2)    var
(3)    x, y: integer;

(4)    function max (i: integer; j: integer): integer;
(5)    {retorna qual é o maior entre os inteiros i e j}
(6)    begin
(7)    if i > j then max := i
(8)    else max := j;
(9)    end;

(10)   begin
(11)   readln (x, y);
(12)   writeln (max (x, y));
(13)   end.
```

Um erro comum de pontuação é o de usar uma vírgula em lugar de ponto-e-vírgula na lista de argumentos de uma declaração de função (por exemplo, usar uma vírgula em lugar do primeiro ponto-e-vírgula à linha (4)); outro é o de omitir um ponto-e-vírgula obrigatório ao final de uma linha (por exemplo, o ponto-e-vírgula ao final da linha (4)); um terceiro é o de colocar um ponto-e-vírgula estranho ao fim de uma linha antes de um else (por exemplo, colocar um ponto-e-vírgula ao fim da linha (7)).

Talvez uma razão pela qual os erros de ponto-e-vírgula sejam tão comuns é que seu uso varia grandemente de uma linguagem para outra. Em Pascal, um ponto-e-vírgula é um separador de enunciados; em C é um terminador. Alguns estudos têm sugerido que a última utilização é menos propensa a erros [AHO].

Um exemplo típico de um erro de operador é o de omitir os dois pontos do símbolo de atribuição “:=”. Erros de grafia em palavras-chave são usualmente raros, mas omitir o i de writeln seria um exemplo representativo.

Muitos compiladores Pascal não têm dificuldades em tratar os erros comuns de inserção, remoção e transformação. De fato, vários compiladores Pascal irão tratar corretamente o programa acima como um erro comum de programação ou de operador; irão emitir somente um diagnóstico de alerta, apontando a construção ilegal.

No entanto, um outro tipo de erro é muito mais difícil de se reparar corretamente. É o caso de um *begin* ou de um *end* ausente (por exemplo, a omissão da linha (9)). A maioria dos compiladores não irá tentar reparar esse tipo de erro.

Como deveria um tratador de erros reportar a presença de um erro? No mínimo, deveria informar o local no programa-fonte onde o mesmo foi detectado, uma vez que existe uma boa

chance de o erro efetivo ter ocorrido uns poucos *tokens* antes.

Uma estratégia comum empregada por muitos compiladores é a de imprimir a linha ilegal com um apontador para a posição na qual o erro foi detectado. Se existir um razoável prognóstico do que o erro realmente foi, uma compreensível mensagem de diagnóstico informativa é também incluída; por exemplo, “ponto-e-vírgula ausente nesta posição”.

Uma vez que o erro tenha sido detectado, como deveria o analisador sintático se recuperar? Como será visto, existe um número de estratégias gerais, mas nenhum método claramente se impõe sobre os demais. Na maioria dos casos, não é adequado para o analisador sintático encerrar logo após detectar o primeiro erro, porque o processamento da entrada restante ainda pode revelar outros. Usualmente, existe alguma forma de recuperação de erros na qual o analisador tenta restaurar a si mesmo para um estado onde o processamento da entrada possa continuar com uma razoável esperança de que o resto correto da entrada será analisado e tratado adequadamente pelo compilador.

Um trabalho inadequado de recuperação pode introduzir uma avalanche de erros “espúrios”, que não foram cometidos pelo programador, mas introduzidos pelas modificações no estado do analisador sintático durante a recuperação de erros. Numa forma similar, uma recuperação de erros sintáticos pode introduzir erros semânticos espúrios que serão detectados posteriormente pelas fases de análise semântica e de geração de código.

Por exemplo, ao se recuperar de um erro, o analisador pode pular a declaração de alguma variável, digamos “zap”. Quando “zap” for posteriormente encontrada nas expressões, não haverá nada sintaticamente errado, mas como não há uma entrada na tabela de símbolos para “zap”, a mensagem “zap não definido” será gerada.

Uma estratégia cautelosa para o compilador é a de inibir as mensagens de erro que provenham de erros descobertos muito proximamente no fluxo de entrada. Em alguns casos, pode haver erros demais para o compilador continuar um processamento sensível (por exemplo, como deveria um compilador Pascal responder ao receber um programa Fortran como entrada?). Parece que uma estratégia de recuperação de erros tem que ser um compromisso cuidadosamente considerado levando em conta os tipos de erros que são mais propensos a ocorrer e razoáveis de processar.

Conforme mencionado, alguns compiladores tentam reparar os erros, num processo em que tentam adivinhar o que o programador queria escrever. Exceto, possivelmente, num ambiente de pequenos programas escritos por estudantes principiantes, a reparação extensiva de erros não é propensa a pagar o seu custo. De fato, com a ênfase crescente na computação interativa e bons ambientes de programação, a tendência parece estar na direção de mecanismos simples de recuperação de erros [AHO].

3.3.1 Estratégias de Recuperação de Erros

Existem muitas estratégias gerais diferentes que um analisador sintático pode empregar para se recuperar de um erro sintático. Apesar de nenhuma delas ter provado ser universalmente aceitável, uns poucos métodos têm ampla aplicabilidade. Introduzimos aqui as seguintes estratégias:

- a) Modalidade do desespero;
- b) Nível de frase;
- c) Produções de erro;
- d) Correção global.

- Recuperação na modalidade do desespero

Este é o método mais simples de implementar e pode ser usado pela maioria dos métodos de análise sintática. Ao descobrir um erro, o analisador sintático descarta símbolos de entrada, um de cada vez, até que seja encontrado um *token* pertencente a um conjunto designado de *tokens* de sincronização. Os *tokens* de sincronização são usualmente delimitadores, tais como o ponto-e-vírgula ou o **end**, cujo papel no programa-fonte seja claro.

Naturalmente, o projetista do compilador precisa selecionar os *tokens* de sincronização apropriados à linguagem-fonte. A correção na modalidade do desespero, que frequentemente pula uma parte considerável da entrada sem verificá-la, procurando por erros adicionais, possui a vantagem da simplicidade e, diferentemente dos outros métodos a serem enfocados adiante, tem a garantia de não entrar num laço infinito. Nas situações em que os erros múltiplos num mesmo enunciado sejam raros, esse método pode ser razoavelmente adequado [AHO].

- Recuperação de frases

Ao descobrir um erro, o analisador sintático pode realizar uma correção local na entrada restante. Isto é, pode substituir um prefixo de entrada remanescente por alguma cadeia que permita ao analisador seguir em frente. Correções locais típicas seriam substituir uma vírgula por um ponto-e-vírgula, remover um ponto-e-vírgula estranho ou inserir um ausente. A escolha da correção local é deixada para o projetista do compilador. Naturalmente, devemos ser cuidadosos, escolhendo substituições que não levem a laços infinitos, como seria o caso, por exemplo, se inseríssemos para sempre na entrada algo à frente do seu símbolo corrente.

Esse tipo de substituição pode corrigir qualquer cadeia, e tem sido usado em vários compiladores de correção de erros. O método foi primeiramente usado na análise sintática *top-down*. Sua maior desvantagem está na dificuldade que tem ao lidar com situações nas quais o erro efetivo ocorreu antes do ponto de detecção.

- Produções de erro

Se todos tivessem uma boa ideia dos erros comuns que poderiam ser encontrados, poder-se-ia aumentar a gramática para a linguagem em exame com as produções que gerassem construções ilegais. Usar-se-ia, então, a gramática aumentada com essas produções de erro para construir um analisador sintático. Se uma produção de erro for usada pelo analisador, podem-se gerar diagnósticos apropriados para indicar a construção ilegal que foi reconhecida na entrada.

- Correção global

Existem algoritmos para escolher uma sequência mínima de mudanças de forma a se obter uma correção global de menor custo. Dada uma cadeia de entrada incorreta x e uma gramática G , esses algoritmos irão encontrar uma árvore gramatical para uma cadeia relacionada y , de tal forma que as inserções, remoções e mudanças de *tokens* requeridas para transformar x em y sejam tão pequenas quanto possível. Infelizmente, esses métodos são em geral muito custosos de implementar, em termos de tempo e espaço e, então, essas técnicas são correntemente apenas de interesse teórico [AHO].

Apesar de tudo, a noção de correção de custo mínimo fornece um padrão de longo alcance para avaliar as técnicas de recuperação de erros e tem sido usada para encontrar cadeias ótimas de substituição para a recuperação em nível de frase.

3.4 ANALISADOR SEMÂNTICO

O compilador executa ainda a análise semântica³. O analisador semântico utiliza a árvore sintática determinada pelo analisador sintático para: identificar operadores e operandos das expressões, reconhecer erros semânticos, fazer verificações de compatibilidade de tipo, analisar o escopo das variáveis, fazer verificações de correspondência entre parâmetros atuais e formais.

Por exemplo, para o comando de atribuição $SOMA := SOMA + 35$, é necessário fazer a seguinte análise:

- O identificador $SOMA$ foi declarado? em caso negativo, erro semântico.
- O identificador $SOMA$ é uma variável? em caso negativo, erro semântico.
- Qual o escopo da declaração da variável $SOMA$: local ou global?
- Qual o tipo da variável $SOMA$? o valor atribuído no lado direito do comando de atribuição é compatível?

Fundamentalmente, a análise semântica trata os aspectos sensíveis ao contexto da sintaxe das linguagens de programação. Por exemplo, não é possível representar em uma gramática livre de contexto uma regra como "Todo identificador deve ser declarado antes de ser usado.", e a verificação de que essa regra foi aplicada cabe à análise semântica. A regra pode ser representada em uma gramática sensível ao contexto, mas não existem algoritmos rápidos para essa classe de gramáticas.

A ideia fundamental é a de usar tabelas externas ao processo de análise sintática, em que as informações são coletadas para posterior consulta. Por exemplo, uma *tabela de símbolos* (ou tabela de identificadores) pode guardar informações sobre as declarações dos identificadores, e essas informações podem ser consultadas para verificar a correção de cada uso de um identificador.

Esse processo é implementado de forma *dirigida pela sintaxe*: associa-se a cada regra da gramática uma ação (*ação semântica*) a ser executada quando o analisador sintático sinaliza um uso da regra. Essas ações são frequentemente implementadas como chamadas de *rotinas semânticas*, e podem ser responsáveis por efetuar a análise semântica e a geração de código, pelo menos parcialmente.

³ A semântica nesse caso engloba apenas uma pequena parte do que vem a ser realmente a semântica de um programa.

Toda vez que o analisador sintático indicar o uso de uma regra associada a uma declaração, a rotina semântica associada a essa regra é chamada para acrescentar à tabela o identificador correspondente, fornecido pelo analisador léxico. Quando uma regra associada a um uso de um identificador for sinalizada, a rotina semântica correspondente será chamada para verificar se o identificador (novamente fornecido pelo analisador léxico) consta da tabela.

Não existe uma fronteira definida entre o que deve ser tratado pelo analisador sintático e o que deve ser tratado pelo analisador semântico, cabendo ao programador do compilador a escolha, segundo suas preferências.

Alguns compiladores incluem a geração de uma representação intermediária para o programa fonte. Uma representação intermediária é um código para uma máquina abstrata e deve ser fácil de produzir e traduzir no programa objeto.

Por exemplo, pode ser usada como forma intermediária o código de três endereços (AHO et. al., 1995). O código de três endereços consiste em uma sequência de instruções, cada uma possuindo no máximo três operandos. Para o comando de atribuição `SOMA := SOMA + 35` tem-se:

`temp1 := 35`

`temp2 := SOMA + temp1`

`SOMA := temp2`

4 MÁQUINA DE TURING

Na disciplina de Compiladores, o conceito de Máquina de Turing (MT) é importante para entender os limites do que pode ser computado e, conseqüentemente, o que pode ou não ser analisado ou transformado por compiladores. Embora o foco principal da disciplina não seja aprofundar os detalhes formais das Máquinas de Turing, é essencial conhecer os tipos básicos que existem, pois eles influenciam no entendimento sobre gramáticas, linguagens formais e autômatos.

4.1 UM BREVE HISTÓRICO SOBRE ALAN TURING

Alan Mathison Turing, nasceu em Londres no dia 23 de junho de 1912, e faleceu em 7 de junho de 1954. Foi um matemático, lógico, criptoanalista e cientista da computação. Foi influente no desenvolvimento da ciência da computação e na formalização do conceito de algoritmo e computação com a Máquina de Turing (A-Machine ou Automatic Machine), desempenhando um papel importante na criação do computador moderno. Foi também pioneiro na inteligência artificial e na ciência da computação (O famoso “Teste de Turing”, o “Jogo da Imitação”).

Aos 24 anos, Turing se consagrou com a projeção de uma máquina - a Máquina de Turing - que pudesse realizar operações computacionais de acordo com um sistema formal. Mostrou como um simples sistema automático poderia manipular símbolos de um sistema de regras próprias. Sua máquina teórica mostrou que sistemas poderosos poderiam ser construídos.

Em 1936, Alan Turing publicou o artigo “*On computable numbers*”, onde sua contribuição foi fundamental para o desenvolvimento de ideias que ocorreriam antes do computador propriamente dito tornar-se realidade. Os cientistas da época admitiam que a matemática não era uma arte misteriosa, e sim uma ciência inteiramente relacionada com regras lógicas. Se uma máquina recebesse essas regras e o problema a ser solucionado, ela seria capaz de resolvê-lo. No entanto, os esforços dos mais competentes matemáticos foram inúteis para desenvolver tal máquina. Turing decidiu examinar o impasse de outra maneira. Verificou os tipos de problemas que uma máquina poderia resolver seguindo regras lógicas, e tentou fazer uma lista de todos eles.

Durante a Segunda Guerra Mundial, Turing trabalhou para a inteligência britânica em um centro especializado em quebra de códigos (retratado no filme “O jogo da Imitação”). Por um tempo ele foi chefe da seção responsável pela criptoanálise da frota naval alemã, onde planejou uma série de técnicas para quebrar os códigos alemães, incluindo uma máquina eletromecânica que poderia encontrar definições para a máquina *Enigma*.

Após a guerra, trabalhou no Laboratório Nacional de Física do Reino Unido, onde criou um dos primeiros projetos para um computador com um programa armazenado, o ACE, além de colaborar no projeto do primeiro computador eletrônico, o Eniac (*Electrical Numerical Integrator and Calculator*), desenvolvido na Universidade da Pensilvânia desde 1943.

A homossexualidade de Turing resultou em um processo criminal em 1952, pois atos homossexuais eram ilegais no Reino Unido na época, onde ele aceitou o tratamento com hormônios femininos e castração química, como alternativa à prisão. Turing morreu em 1954, algumas semanas antes de seu aniversário de 42 anos, devido a um aparente auto administrado envenenamento por cianeto, apesar de sua mãe (e alguns outros) terem considerado sua morte acidental.

Em 10 de setembro de 2009, após uma campanha de internet, o primeiro-ministro britânico Gordon Brown fez um pedido público oficial de desculpas, em nome do governo britânico, devido à maneira pela qual Turing foi tratado após a guerra. Somente em 24 de dezembro de 2013, Alan Turing recebeu o perdão real da rainha Elizabeth II, da condenação por homossexualidade.

4.2 A MÁQUINA DE TURING

a. Máquina de Turing Determinística (MTD)

Para cada combinação de estado atual e símbolo lido, existe no máximo **uma ação possível** (escrever símbolo, mover a fita, mudar de estado). A MTD é o modelo clássico da MT, usado como referência para definir o que é computável. Ele serve de base para entender as linguagens recursivamente enumeráveis, o tipo mais geral de linguagem – o **tipo 0** da hierarquia de Chomsky.

b. Máquina de Turing Não-Determinística (MTND)

Permite múltiplas ações possíveis para um mesmo par (estado, símbolo). Útil em teoria da complexidade.

c. Máquina de Turing Multifaixa (ou multifita)

Possui múltiplas fitas com cabeças independentes. Melhora a eficiência, mas não o poder computacional.

d. Máquina de Turing Bidirecional

Fita infinita em ambas as direções. Mais flexível, mas equivalente à MT tradicional.

e. Máquina de Turing Multidimensional (ou bidimensional)

Fita organizada em grade 2D ou mais dimensões. Não aumenta poder computacional.

f. Máquina de Turing Universal

Simula qualquer outra MT. Inspiração para o funcionamento de compiladores, porque o compilador pode ser visto como uma "máquina universal" que traduz programas escritos em uma linguagem para outra (por exemplo, código fonte para código de máquina).

g. Máquina de Turing com ou sem parada

Algumas MTs entram em loop infinito, enquanto outras sempre chegam a um estado de parada. Esta nomenclatura está relacionada ao Problema da Parada, que é indecidível.

Importante:

Esta apostila adota a Máquina de Turing Determinística como modelo de referência para o estudo dos conceitos fundamentais introduzidos por Alan Turing. Isso se deve ao fato de que, em termos de poder computacional, a MTD é equivalente a outras variantes de Máquina de Turing, sendo, portanto, suficiente para representar e resolver todos os problemas computáveis reconhecíveis por esses modelos.

4.3 DEFINIÇÃO DE MÁQUINA DE TURING

A **Máquina de Turing** para ser considerada como um modelo formal de procedimento efetivo, algoritmo ou função computável deve satisfazer às seguintes propriedades, entre outras:

- a) A descrição do algoritmo deve ser finita.
- b) Deve consistir de passos:
 - discretos (claros, não ambíguos);
 - executáveis mecanicamente;
 - em um tempo finito.

O modelo proposto por Alan Turing em 1936, conhecido como Máquina de Turing, consiste basicamente de 3 partes:

- a) **Fita**: usada simultaneamente como dispositivo de entrada, saída e memória de trabalho;
- b) **Unidade de Controle**: reflete o estado corrente da máquina. Possui uma unidade de leitura e gravação (cabeça da fita) a qual acessa uma célula da fita de cada vez e movimenta-se para a esquerda ou direita;
- c) **Programa ou Função de Transição**: função que comanda as leituras e gravações, o sentido de movimento da cabeça da fita e define o estado da máquina.

Por exemplo:

$$(q_0, a) \rightarrow (q_1, A, E)$$

A fita é finita à esquerda e infinita à direita, sendo dividida em células, onde cada uma armazena um símbolo. Os símbolos podem pertencer ao alfabeto de entrada, ao alfabeto auxiliar ou ainda, ser branco ou marcador de início de fita.

Inicialmente, a palavra a ser processada (ou seja, a informação de entrada para a máquina) ocupa as células mais à esquerda, após o marcador de início de fita, ficando as demais com “branco”, como ilustrado na Figura a seguir, onde β e $\bullet$ representam “branco” e “marcador de início de fita”, respectivamente.

A unidade de controle (cabeça da fita) possui um número finito e predefinido de estados. A cabeça da fita lê o símbolo de uma célula de cada vez e grava um novo símbolo. Após a leitura ou gravação, a cabeça move uma célula para a direita ou esquerda. O símbolo gravado e o sentido do movimento são definidos pelo programa.

O programa é uma função que, dependendo do estado corrente da máquina e do símbolo lido, determina:

- o símbolo a ser gravado;
- o sentido do movimento da cabeça;
- o novo estado.

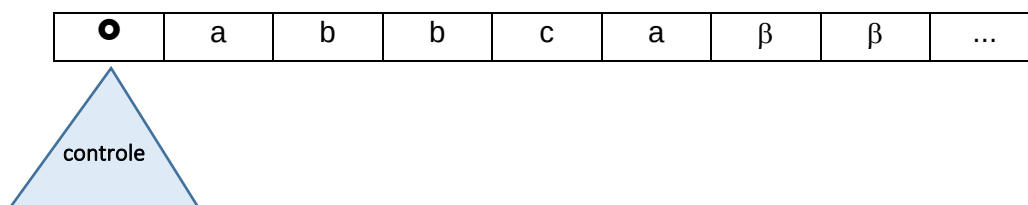


FIGURA 18: FITA E UNIDADE DE CONTROLE DE UMA MÁQUINA DE TURING

4.4 DEFINIÇÃO FORMAL DA MÁQUINA DE TURING

Uma Máquina de Turing é uma ócupla: $M = (\Sigma, Q, \delta, q_0, F, V, \beta, \bullet)$

Onde:

Σ = alfabeto de símbolos de entrada;

Q = conjunto de estados possíveis da máquina o qual é finito;

δ = programa ou função de transição: $\delta: Q \times (\Sigma \cup V \cup \{\beta, \bullet\}) \rightarrow Q \times (\Sigma \cup V \cup \{\beta, \bullet\}) \times \{E, D\}$ a qual é uma função parcial;

q_0 = estado inicial da máquina tal que q_0 é elemento de Q ;

F = conjunto de estados finais tal que F está contido em Q ;

V = alfabeto auxiliar (pode ser vazio);

β = símbolo especial branco;

$\bullet$ = símbolo ou marcador de início da fita.

O símbolo de início de fita sempre ocorre exatamente uma vez e na célula mais à esquerda da fita, auxiliando na identificação de que a cabeça de leitura/gravação se encontra na célula mais à esquerda.

A função programa, considera o estado corrente e o símbolo lido da fita para determinar o novo estado, o símbolo a ser gravado e o sentido do movimento da cabeça; onde esquerda e direita são representados por **E** e **D**, respectivamente.

A função programa pode ser interpretada como um grafo finito direto, como ilustrado na Figura a seguir. Neste caso, os estados iniciais e finais são representados como nos Autômatos Finitos.

O processamento de uma Máquina de Turing $M = (\Sigma, Q, \delta, q_0, F, V, \beta, \bullet)$, para uma palavra de entrada **w**, consiste na sucessiva aplicação da função programa a partir do estado inicial **q₀** e da cabeça posicionada na célula mais à esquerda da fita até ocorrer uma condição de parada.

O processamento de **M** para a entrada **w**, pode parar ou ficar em *loop* infinito. As condições de parada são as seguintes:

- A máquina assume um estado final: a máquina para e a palavra de entrada é aceita.
- A função programa é indefinida para o argumento (símbolo lido e estado corrente): a máquina para e a palavra de entrada é rejeitada;
- O argumento corrente da função programa define um movimento à esquerda e a cabeça da fita já se encontra na célula mais à esquerda: a máquina para e a palavra de entrada é rejeitada.

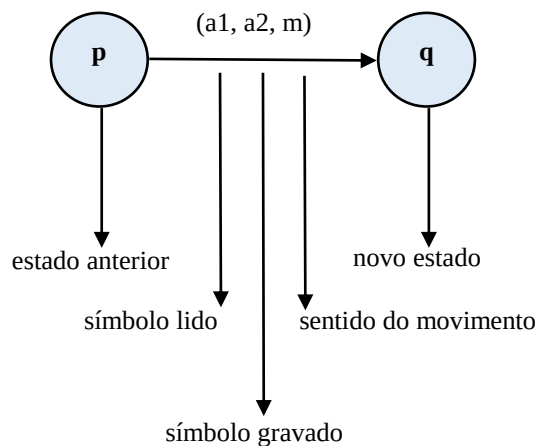


FIGURA 19: REPRESENTAÇÃO DA FUNÇÃO PROGRAMA COMO UM GRAFO

4.5 LINGUAGENS RECONHECIDAS POR MÁQUINA DE TURING

As linguagens reconhecidas por Máquinas de Turing são chamadas de **Linguagens Recursivamente Enumeráveis**. Entre elas, algumas podem ser sempre decididas pela máquina, enquanto outras só podem ser reconhecidas — ou seja, a máquina pode não conseguir dizer quando uma palavra não pertence à linguagem.

Logo:

Turing-Decidível (ou recursiva): existe MT que a decide, ou seja, a máquina decide se $w \in L$ ou $w \notin L$.

Turing-Reconhecível (ou recursivamente enumerável): existe MT que a reconhece, ou seja, a MT aceita se $w \in L$ e rejeita ou entra em loop se $w \notin L$.

Note que as linguagens recursivamente enumeráveis são aquelas geradas pelas gramáticas irrestritas, também conhecidas como gramáticas do Tipo 0 na Hierarquia de Chomsky.

4.6 MÁQUINA DE TURING DETERMINÍSTICA

Exemplo 1: Máquina de Turing – Duplo Balanceamento.

Considere a linguagem:

$$L_1 = \{a^n b^n \mid n \geq 0\}$$

A Máquina de Turing:

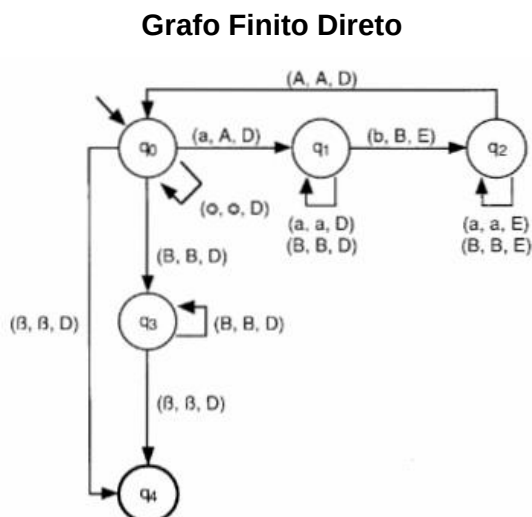


FIGURA 20: MÁQUINA DE TURING

$$M_1 = (\{a, b\}, \{q_0, q_1, q_2, q_3, q_4\}, \delta_1, q_0, \{q_4\}, \{A, B\}, \beta, \bullet)$$

Ilustrada na Figura acima e onde δ_1 é como na tabela a seguir, é tal que:

$$\text{ACEITA}(M_1) = L_1$$

$$\text{REJEITA}(M_1) = \Sigma^* - L_1$$

e, portanto, $\text{LOOP}(M_1) = \emptyset$ (ou seja, para L_1 , a máquina apresentada nunca entrará em *looping*)

δ_1	$\bullet$	a	b	A	B	β
q_0	$(q_0, \bullet, D)$	(q_1, A, D)	-	-	(q_3, B, D)	(q_4, β, D)
q_1	-	(q_1, a, D)	(q_2, B, E)	-	(q_1, B, D)	-
q_2	-	(q_2, a, E)	-	(q_0, A, D)	(q_2, B, E)	-
q_3	-	-	-	-	(q_3, B, D)	(q_4, β, D)
q_4^*	-	-	-	-	-	-

O algoritmo apresentado reconhece o primeiro símbolo **a**, o qual é marcado com **A**, e movimenta a cabeça da fita à direita, procurando o **b** correspondente, o qual é marcado como **B**. Este ciclo é repetido sucessivamente até identificar para cada **a** o seu correspondente **b**.

Adicionalmente, o algoritmo garante que qualquer outra palavra que não esteja na forma $a^n b^n$ é rejeitada. Nota-se que o símbolo de início de fita não tem influência na solução proposta.

A Figura a seguir ilustra a sequência do processamento da Máquina de Turing M_1 para a entrada $w = aabb$.

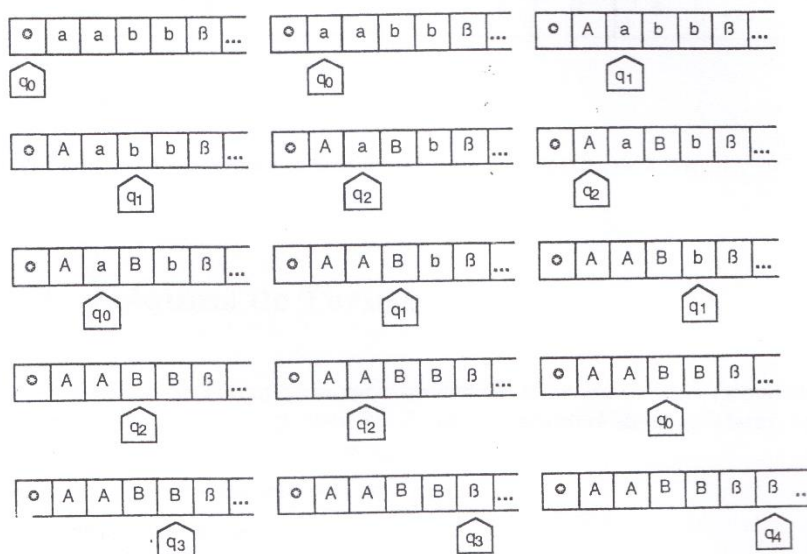


FIGURA 21: SEQUÊNCIA DE PROCESSAMENTO DE UMA MÁQUINA DE TURING

4.6.1 Exercícios

1) Resolva as questões propostas para a Máquina de Turing e a Linguagem em questão.

$L = a^x b^y \mid x, y \geq 1 \text{ e } y = 2x \text{ ou } a^n b^{2n} \mid n \in \mathbb{N}^* \text{ (naturais não nulos)}$

δ	$\bullet$	a	b	x	y	β
$\rightarrow q_0$	$(q_0, \bullet, D)$	(q_0, a, D)	(q_1, Y, D)	-	-	-
q1	-	-	(q_2, Y, E)	-	-	-
q2	-	(q_3, X, D)	-	(q_2, X, E)	(q_2, Y, E)	-
q3	-	-	(q_1, Y, D)	(q_3, X, D)	(q_3, Y, D)	(q_4, β, E)
q4	$(q_5, \bullet, D)$	-	-	(q_4, X, E)	(q_4, Y, E)	-
*q5	-	-	-	-	-	-

Entradas:

- a) abb
- b) aabbbb
- c) aabb

Após resolver as entradas acima, descreva com suas palavras como funciona o algoritmo desta máquina.

2) Resolva as questões propostas para a Máquina de Turing e a Linguagem em questão.

$L = \{(a|b)^* \text{ e a quantidade de } a\text{'s é igual a quantidade de } b\text{'s}\}$

δ	$\bullet$	a	b	x	β
$\rightarrow q_0$	$(q_0, \bullet, D)$	(q_2, X, D)	(q_1, X, D)	(q_0, X, D)	(q_4, β, D)
q1	-	(q_3, X, E)	(q_1, b, D)	(q_1, X, D)	-
q2	-	(q_2, a, D)	(q_3, X, E)	(q_2, X, D)	-
q3	-	(q_3, a, E)	(q_3, b, E)	(q_0, X, D)	-
*q4	-	-	-	-	-

Entradas:

- a) ababba
- b) abbaa
- c) abbbbaa

Após resolver as entradas acima, descreva com suas palavras como funciona o algoritmo desta máquina.

3) Resolva as questões propostas para a Máquina de Turing e a Linguagem em questão.

$L = \{w | w \in (0,1)^* \text{ e } w \text{ forme um número binário par}\}$ OBS: “branco” não é aceito porque não é número binário.

δ	$\bullet$	0	1	β
$\rightarrow q_0$	$(q_0, \bullet, D)$	$(q_0, 0, D)$	$(q_0, 1, D)$	(q_1, β, E)
q1	-	$(q_2, 0, D)$	-	-
*q2	-	-	-	-

Entradas:

- a) 1010
- b) 1011
- c) 11
- d) 10

Após resolver as entradas acima, descreva com suas palavras como funciona o algoritmo desta máquina.

4) Resolva as questões propostas para a Máquina de Turing e a Linguagem em questão.

$L = \{a^n b^n c^n \mid n \geq 1\}$ Observação: Linguagem de triplo balanceamento.

δ	$\bullet$	a	b	c	A	B	C	β
$\rightarrow q_0$	$(q_1, \bullet, D)$	-	-	-	-	-	-	-
q1	-	(q_2, A, D)	-	-	-	(q_5, B, D)	-	(q_6, β, D)
q2	-	(q_2, a, D)	(q_3, B, D)	-	-	(q_2, B, D)	-	-
q3	-	-	(q_3, b, D)	(q_4, C, E)	-	-	(q_3, C, D)	-
q4	-	(q_4, a, E)	(q_4, b, E)	-	(q_1, A, D)	(q_4, B, E)	(q_4, C, E)	-
q5	-	-	-	-	-	(q_5, B, D)	(q_5, C, D)	(q_6, β, E)
*q6	-	-	-	-	-	-	-	-

Entradas:

- a) aabbcc
- b) ccbbaa
- c) abcabc
- d) abc

Após resolver as entradas acima, descreva com suas palavras como funciona o algoritmo desta máquina.

5) Resolva a questão proposta para a Máquina de Turing e a Linguagem em questão.

$L = \{ww^R \mid w \in \{a, b\}^*\}$, onde:

O "R" em ww^R significa que a segunda parte da palavra é o **reverso** da primeira.

δ	$\bullet$	a	b	K	β
q ₀	$(q_0, \bullet, D)$	(q_1, K, D)	(q_4, K, D)	(q_6, K, D)	(q_6, β, D)
q ₁	-	(q_1, a, D)	(q_1, b, D)	(q_2, K, E)	(q_2, β, E)
q ₂	-	(q_3, K, E)	-	-	-
q ₃	-	(q_3, a, E)	(q_3, b, E)	(q_0, K, D)	-
q ₄	-	(q_4, a, D)	(q_4, b, D)	(q_5, K, E)	(q_5, β, E)
q ₅	-	-	(q_3, K, E)	-	-
q ₆ *	-	-	-	-	-

Verifique as seguintes entradas:

- a) abba
- b) aba
- c) abbbba
- d) abaaba
- e) babba

Descreva o funcionamento da máquina:

6) Considere a seguinte Máquina de Turing abaixo, onde:

A Máquina de Turing abaixo foi criada para comparar duas palavras escritas com as letras **a** e **b**, separadas pelo símbolo **\$**. Ela verifica se as duas palavras são exatamente iguais.

Π	a	b	\$	A	B	β	$\otimes$
q₀	-	-	-	-	-	-	(q ₁ , $\otimes$, D)
q₁	(q ₂ , A, D)	(q ₄ , B, D)	(q ₈ , \$, D)	-	-	-	-
q₂	(q ₂ , a, D)	(q ₂ , b, D)	(q ₃ , \$, D)	-	-	-	-
q₃	(q ₆ , A, E)	-	-	(q ₃ , A, D)	(q ₃ , B, D)	-	-
q₄	(q ₄ , a, D)	(q ₄ , b, D)	(q ₅ , \$, D)	-	-	-	-
q₅	-	(q ₆ , B, E)	-	(q ₅ , A, D)	(q ₅ , B, D)	-	-
q₆	-	-	(q ₇ , \$, E)	(q ₆ , A, E)	(q ₆ , B, E)	-	-
q₇	(q ₇ , a, E)	(q ₇ , b, E)	-	(q ₁ , A, D)	(q ₁ , B, D)	-	-
q₈	-	-	-	(q ₈ , A, D)	(q ₈ , B, D)	(q ₉ , β , E)	-
* q₉	-	-	-	-	-	-	-

Verifique as seguintes entradas:

- a) abb\$abb
- b) abb\$bba
- c) aabb\$aabb

Descreva o funcionamento da máquina:

5 TRANSFORMAÇÕES EM GLC'S

Uma vez que existem diferentes métodos de análise, cada qual exigindo gramáticas com características específicas, é importante que uma gramática possa ser transformada, porém, sem perder a qualidade de gerar a mesma linguagem.

As gramáticas que, mesmo tendo conjuntos diferentes de produções geram a mesma linguagem são ditas gramáticas equivalentes.

Os métodos de reconhecimento sintático podem exigir uma ou mais das seguintes transformações: (a) eliminação de produções vazias, (b) retirada de recursividade à esquerda e (c) fatoração de produções. Estes três tipos de transformações são apresentados a seguir.

5.1 ELIMINAÇÃO DE ε -PRODUÇÕES (PRODUÇÕES VAZIAS)

Uma GLC pode ter produções do tipo $A ::= \varepsilon$. Mas toda GLC pode ser transformada em uma GLC equivalente sem este tipo de produções, com exceção da produção $S ::= \varepsilon$. Se esta existir (ou seja, a cadeia vazia pertence à linguagem).

Assim procedendo, é possível mostrar que toda GLC pode obedecer à restrição das GSC (tipo 1). É o método da eliminação de ε -produções, da qual se tratará agora.

A exclusão de ε -produções pode determinar modificações diversas nas produções da gramática. O algoritmo é dividido em três etapas, como segue:

a) *Variáveis que constituem produções vazias.* Considera, inicialmente, todas as variáveis que geram diretamente ε (ex: $A \rightarrow \varepsilon$). A seguir, sucessivamente são determinadas as variáveis que indiretamente geram ε (ex: $B \rightarrow A$);

b) *Exclusão de produções vazias.* Inicialmente, são consideradas todas as produções não-vazias. A seguir, cada produção cujo lado direito possui uma variável que gera a palavra vazia, determina uma produção adicional, sem esta variável;

c) *Inclusão de geração da palavra vazia, se necessário.* Se a palavra vazia pertence à linguagem, então é incluída uma produção para gerar a palavra vazia.

O método consiste em determinar, para cada variável A em N , se $A \rightarrow^* \varepsilon$. Se isto for verdade, se diz que a variável A é anulável. Pode-se assim substituir cada produção da forma $B ::= X_1X_2...X_n$ por todas as produções formadas pela retirada de uma ou mais variáveis X_i anuláveis.

Exemplo:

$A ::= BCDe$
 $B ::= \varepsilon \mid e$
 $C ::= \varepsilon \mid a$
 $D ::= b \mid cC$

Neste caso, as variáveis anuláveis são B e C , assim a gramática pode ser transformada na seguinte:

$A ::= BCDe \mid CDe \mid BDe \mid De$
 $B ::= e$
 $C ::= a$
 $D ::= b \mid c \mid cC$

Os não-terminais que derivam a sentença ε são chamados de ε -não-terminais. Um não-terminal A é um ε -não-terminal se existir uma derivação $A \rightarrow^* \varepsilon$ em G . Note que ε está em $L(G)$ se e somente se S é um ε -não-terminal. Se G não tem ε -não-terminais, ela é dita ε -livre.

Se G tem ε -produções, então em uma derivação sentencial da forma: $S \rightarrow \alpha_1 \rightarrow \alpha_2 \rightarrow \dots \rightarrow \alpha_n$, os tamanhos das sentenças irão variar não-monotonicamente um em relação ao outro, isto é, ora aumentam, ora diminuem. Entretanto, se G não tem ε -produções, então:

$$|S| \leq |\alpha_1| \leq |\alpha_2| \leq \dots \leq |\alpha_n|$$

isto é, os tamanhos das sentenças são monotonicamente crescentes. Esta propriedade é útil quando se quer testar se uma dada palavra é ou não gerada por uma GLC.

💡 **Dica : Como eliminar as ε -produções.**

- 1º Se o símbolo inicial S possui ε -produção, cria a seguinte regra: $S' ::= S / \varepsilon$
- 2º Riscar as ε -produções de todas as regras
- 3º marcar os **NT** (não-terminais) que possuem ε -produções indiretas, ou seja, muda para um estado onde tenha ε -produções.
- 4º Onde houver **NT** marcados, fazer novas combinações, criando novas produções.

5.2 ELIMINAÇÃO DE RECURSÃO À ESQUERDA

Uma gramática é *recursiva à esquerda* se possui um não-terminal A tal que exista uma derivação $A^+ \rightarrow A\alpha$ para alguma cadeia α . Os métodos de análise sintática *top-down* não podem processar gramáticas recursivas à esquerda e, conseqüentemente, uma transformação que elimine a recursão à esquerda é necessária.

Ex: Considere a seguinte gramática para as expressões aritméticas.

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Eliminando a *recursão imediata à esquerda* (produções da forma $A \rightarrow A\alpha$) nas produções para E e T , obtém-se:

$$\begin{aligned} E &\rightarrow TE' \\ E' &\rightarrow + TE' \mid \epsilon \\ T &\rightarrow FT' \\ T' &\rightarrow * FT' \mid \epsilon \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Para os casos de recursividade direta, a recursividade pode ser eliminada manualmente. Para recursões indiretas, a transformação é mais trabalhosa e exige que a gramática seja primeiramente simplificada.

Um não-terminal A , em uma GLC $G=(N,T,P,S)$ é *recursivo* se $A \rightarrow \alpha A \beta$, para α e $\beta \in (N \cup T)^*$.

Se $\alpha = \epsilon$, então A é *recursivo à esquerda*; se $\beta = \epsilon$, então A é *recursivo à direita*. Esta recursividade pode ser direta ou indireta.

Uma gramática com pelo menos um não-terminal recursivo à esquerda ou à direita é uma *gramática recursiva à esquerda* ou *à direita*, respectivamente.

Uma GLC $G = (N,T,P,S)$ possui *recursão à esquerda direta* se P contém pelo menos uma produção da forma $A ::= A\alpha$.

Uma GLC $G = (N,T,P,S)$ possui *recursão à esquerda indireta* se existe em G uma derivação da forma $A \rightarrow^n A\beta$, para algum $n \geq 2$.

Para eliminar as recursões diretas à esquerda nas produções:

$$A ::= A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \dots \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$$

onde nenhum β_i começa com A , deve-se substituir estas produções pelas seguintes :

$$\begin{aligned} A &::= \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A' \\ A' &::= \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon \end{aligned}$$

onde A' é um novo não-terminal.

Exemplo Eliminação de recursão à esquerda, dada a seguinte gramática:

$A ::= Aa \mid cd \mid Adc$
 $B ::= ab \mid bA \mid Ba$

O resultado fica:

$A ::= cdA'$
 $A' ::= aA' \mid dcA' \mid \varepsilon$
 $B ::= abB' \mid bAB'$
 $B' ::= aB' \mid \varepsilon$

A seguir é apresentado um algoritmo para eliminar recursões diretas e indiretas à esquerda:

Algoritmo: Eliminar Recursões à Esquerda

Entrada: Uma GLC $G=(N,T,P,S)$.

Saída: Uma GLC $G=(N',T,P',S)$ sem recursão à esquerda.

$N' := N$;

$P' := P$;

Ordene os não-terminais de N' em uma ordem qualquer (por exemplo, $A_1, A_2, A_3, \dots, A_n$);

Para i de 1 até n , faça:

 Para j de 1 até $i-1$ faça

 Substitua as produções de P' da forma $A_i ::= A_j\gamma$ por produções da forma

$A_i ::= \delta_1\gamma \mid \delta_2\gamma \mid \dots \mid \delta_k\gamma$, onde $\delta_1, \delta_2, \dots, \delta_k$ são os lados direitos das produções com lado esquerdo A_j , ou seja, $A_j ::= \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$.

 Fim Para;

 Elimine as recursões diretas das produções de P' com lado esquerdo A_i ;

Fim Para;

Note que este algoritmo já elimina as recursões à esquerda diretas e indiretas.

☛ **Dica: Eliminação de recursão à esquerda.**

1º Detectar recursão à esquerda

2º Eliminar a recursão, mantendo as não recursivas na mesma regra com X' no final de cada produção

3º Criar um estado ' X ' com produções formadas pelas produções recursivas mais ' X ' no final de cada produção. Adicionar no final a produção ε

4º Verificar se não existem recursões indiretas

5.3 FATORAÇÃO À ESQUERDA

A fatoração à esquerda é uma transformação gramatical útil para a criação de uma gramática adequada à análise sintática preditiva, que é vista na sequência deste material. A ideia básica está no seguinte: quando não estiver claro qual das duas produções alternativas usar para expandir um não-terminal A , estaremos capacitados a reescrever as produções- A e postergar a decisão até que tenhamos visto o suficiente da entrada para realizarmos a escolha certa.

Por exemplo, se existirem as duas produções:

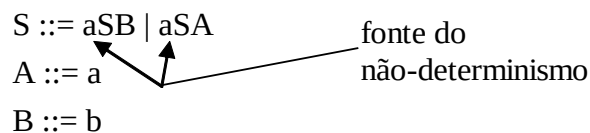
$cmd \rightarrow \text{if expr then cmd else cmd} \mid \text{if expr then cmd}$

Ao visualizar o *token* de entrada **if**, não se pode imediatamente dizer qual produção escolher a fim de expandir cmd . Em geral, se $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ são duas produções- A , e a entrada começa por uma cadeia não vazia derivada a partir de α , não sabemos se vamos expandir A em $\alpha\beta_1$ ou em $\alpha\beta_2$. Entretanto, podemos postergar a decisão expandindo A para $\alpha\beta'$. Então, após enxergarmos a entrada derivada a partir de α , expandimos A' em β_1 ou em β_2 . Isto é, as produções originais, fatoradas à esquerda, se tornam:

$A \rightarrow \alpha A'$
 $A' \rightarrow \beta_1 \mid \beta_2$

Uma GLC está *fatorada* se ela é determinística, ou seja, se ela não possui produções para um mesmo não-terminal no lado esquerdo cujo lado direito inicie com a mesma cadeia de símbolos ou com símbolos que derivam sequências que iniciem com a mesma cadeia de símbolos. De acordo com esta definição, a seguinte gramática é não-determinística:

$S ::= aSB \mid aSA$
 $A ::= a$
 $B ::= b$



A fatoração à esquerda permite eliminar a indecisão sobre qual produção aplicar quando duas ou mais produções iniciam com a mesma forma sentencial. Por exemplo, a indecisão referente às produções:

$A ::= \alpha\beta_1 \mid \alpha\beta_2$

Seria eliminada fatorando as mesmas para:

$A ::= \alpha X$
 $X ::= \beta_1 \mid \beta_2$

Para fatorar uma GLC, deve-se alterar as produções envolvidas no não-determinismo da seguinte forma:

a) As produções com não-determinismo direto da forma $A ::= \alpha\beta \mid \alpha\gamma$ serão substituídas por:

$A ::= \alpha A'$
 $A' ::= \beta \mid \gamma$

Na gramática acima, temos $\alpha = aS$, assim, com a eliminação do não-determinismo, teríamos a seguinte gramática:

```
S ::= aSS'
S' ::= B | A
A ::= a
B ::= b
```

b) Não-determinismo indireto é retirado através de sua transformação em não-determinismo direto (através de derivações sucessivas) e posterior eliminação segundo o item (a). Tomemos por exemplo a seguinte gramática:

```
S ::= AC | BD
A ::= aD | cB
B ::= aB | dD
C ::= eC | eA
D ::= fD | AB
```

Onde o não-determinismo indireto se dá pelo fato de que A e B podem iniciar com o terminal a . Assim, transformaremos primeiro o não-determinismo indireto em não-determinismo direto, substituindo os não-terminais A e B pelas suas produções em S :

```
S ::= aDC | cBC | aBD | dDD
A ::= aD | cB
B ::= aB | dD
C ::= eC | eA
D ::= fD | AB
```

O próximo passo é simples, bastando agora eliminar o não-determinismo direto, segundo a regra do item a, ou seja:

```
S ::= aS' | cBC | dDD
S' ::= DC | BD
A ::= aD | cB
B ::= aB | dD
C ::= eC'
C' ::= C | A
D ::= fD | AB
```

Observação sobre a fatoração:

Na GLC:

```
S ::= Ab | ab | baA
A ::= aab | b
```

Fatora-se:

$S ::= aS' | bS''$ onde:

S' para regras que começam com "a"; e S'' para regras que começam com "b".

☛ **Dica: Fatoração.**

- 1º Eliminar indeterminismo direto.
- 2º Eliminar indeterminismo indireto.
- 3º Fazer substituições para descobrir indeterminismos indiretos.

Obs : fazer 1º os diretos e depois os indiretos.

5.4 ELIMINANDO A AMBIGUIDADE DAS GLC'S

Algumas vezes uma gramática ambígua pode ser reescrita de forma a eliminar a ambiguidade. Como um exemplo, vamos eliminar a ambiguidade da seguinte gramática do “*else-vazio*”:

$cmd \rightarrow \text{if } expr \text{ then } cmd \mid \text{if } expr \text{ then } cmd \text{ else } cmd \mid \text{outro}$

Aqui “**outro**” significa qualquer outro enunciado. De acordo com esta gramática, o enunciado condicional composto:

if E_1 then S_1 else if E_2 then S_2 else S_3

possui a árvore gramatical mostrada na figura abaixo.

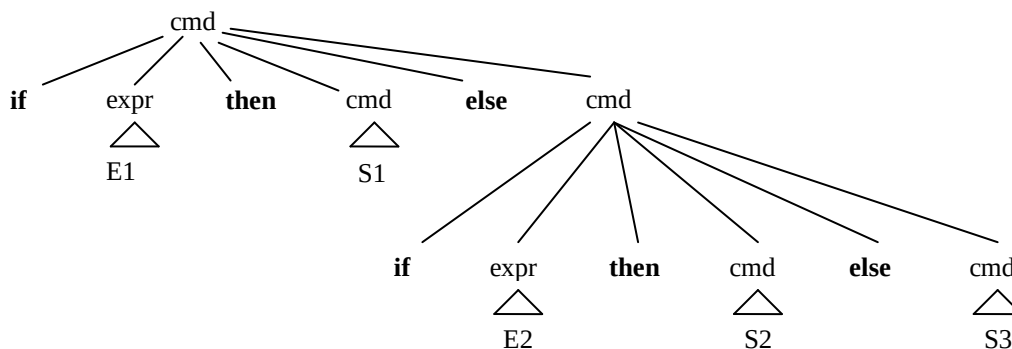


FIGURA 22: ÁRVORE GRAMATICAL PARA O ENUNCIADO CONDICIONAL [AHO].

A gramática listada acima é ambígua, uma vez que a cadeia:

if E_1 then if E_2 then S_1 else S_2

possui as duas árvores mostradas na figura a seguir.

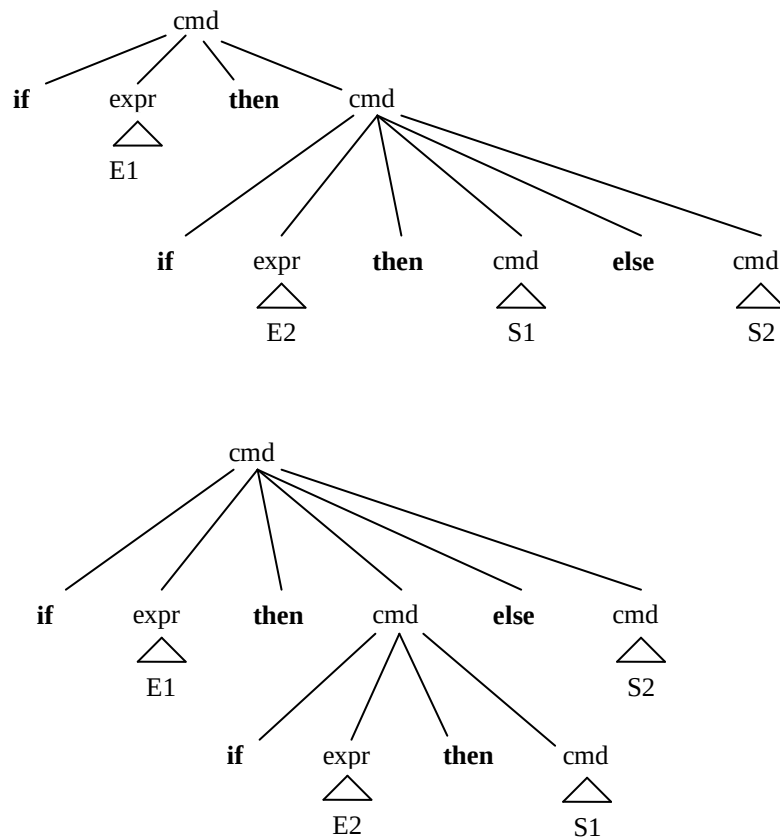


FIGURA 23: DUAS ÁRVORES GRAMATICAIS PARA UMA SENTENÇA AMBÍGUA [AHO].

Em todas as linguagens de programação com enunciados condicionais desta forma, a primeira árvore gramatical é preferida. A regra geral é “associar cada **else** ao **then** anterior mais próximo ainda não associado”. Essa regra de ambiguidade pode ser incorporada diretamente à gramática. Por exemplo, podemos reescrever a gramática do exemplo sob a forma inambígua abaixo.

A ideia está em que um enunciado figurando entre um **then** e um **else** precisa ser “associado”, isto é, não pode terminar com um **then** ainda não associado seguido por qualquer outro enunciado, pois o **else** seria forçado a se associar a esse **then** não associado. Um enunciado associado ou é um enunciado **if-then-else** contendo somente enunciados associados ou é qualquer outro tipo de enunciado incondicional. Dessa forma podemos usar a gramática:

$cmd \rightarrow cmd_associado \mid cmd_não\ associado$

$cmd_associado \rightarrow \text{if } expr \text{ then } cmd_associado \text{ else } cmd_associado \mid \text{outro}$

$cmd_não\ associado \rightarrow \text{if } expr \text{ then } cmd \mid \text{if } expr \text{ then } cmd_associado \text{ else } cmd_não\ associado$

Esta gramática gera o mesmo conjunto de cadeias que a anterior, mas permite somente uma estruturação gramatical, especificamente aquela que associa cada **else** com o **then** anterior mais próximo ainda não associado.

5.5 TRANSFORMAÇÕES EM GLC'S - EXERCÍCIOS PROPOSTOS

1 – Elimine as ϵ -produções das seguintes gramáticas:

a)

$X ::= 0X1 \mid 1X1 \mid AB \mid B1C0$

$A ::= 1A1 \mid 00A \mid ABC \mid \epsilon$

$B ::= BA \mid 1B0 \mid BAC \mid \epsilon$

$C ::= BCB \mid 0011 \mid 1A1 \mid \epsilon$

b)

$S ::= ABC \mid aBC \mid bC$

$A ::= aAa \mid \epsilon$

$B ::= BC \mid bB \mid \epsilon$

$C ::= CC \mid cC \mid \epsilon$

c)

$S ::= 0A1 \mid 00B \mid A0B \mid AC$

$A ::= 1A0 \mid 1A \mid AB \mid \epsilon$

$B ::= BA \mid 00B \mid 11B \mid \epsilon$

$C ::= 0C1 \mid 01$

d)

$S ::= AB \mid CD \mid aAa$

$A ::= aAa \mid \epsilon$

$B ::= bbbB \mid \epsilon$

$C ::= cC \mid \epsilon$

$D ::= dDdd \mid ddd$

e)

$S ::= 1AB \mid 0ABC$

$A ::= 1A0C \mid AC \mid 1 \mid \epsilon$

$B ::= ACA \mid 1B \mid 0$

$C ::= 1C \mid C1C \mid \epsilon$

f)

$S ::= 1B \mid BC \mid 0D$

$B ::= BCB \mid 01 \mid \epsilon$

$C ::= C1 \mid B1S \mid \epsilon$

$D ::= 1D0 \mid 10$

2 – Eliminação de recursão à esquerda :

a)

A ::= Abc | Acd | cdB | c
B ::= Bc | dd

b)

S ::= Aab | Bc | ScAb
A ::= Sac | BaA | ab
B ::= Ac | aBb | ab

c)

A ::= Ac | bd | cd
B ::= ac | Bd | Ac

d)

S ::= Aa | Bb | Sac
A ::= Sab | ab
B ::= a | Ba

e)

S ::= A0 | 0B1 | 11C
A ::= 01 | 0BC | S0
B ::= B01 | 01A
C ::= 01 | 10

f)

S ::= 01S | 00A | S10 | A01
A ::= 00D | 01S | A00 | S11 | 0B
B ::= B00 | B11 | C01 | 001
C ::= B10 | C00 | 01C | 01B | 0
D ::= D01 | D00 | 01 | 00

g)

S ::= SaB | Sbc | Acd
A ::= ABc | Acd | Bcd | Cd
B ::= Acd | Bcc | Cdc
C ::= Ac | Cd | dc

h)

S ::= Cab | Ab | b
A ::= Bcd | Ac | ab
B ::= Aca | Bc | Cba | a
C ::= DaC | Cc | c
D ::= ac | CaD

3 – Fatoração :

a)
C ::= V = exp | id (E)
V ::= id [E] | id
E ::= exp + E | exp

b)
S ::= bcD | Bcd
B ::= bB | b
D ::= dD | d

c)
S ::= aA | aB
A ::= aA | a
B ::= b

d)
S ::= Ab | ab | baA
A ::= aab | b

e)
S ::= Abc | bBC | bCD
A ::= aBC | aDC
B ::= dCc | dc
C ::= Acd | cd
D ::= aBC | abc | aC

f)
S ::= 1A0 | 1B1
A ::= 1A0 | 100
B ::= C10 | C01
C ::= 11C | 11D
D ::= 10 | 11

g)
S ::= 10D | 11C | 0B
B ::= 1CD | 101B | 01
C ::= 101C | 1B0 | 00
D ::= 00B | 011D | 110

h)
S ::= aBd | acD | bC
B ::= bDc | bCd | ad
D ::= cdD | caB
C ::= cbB | adD

Bibliografia

BIBLIOGRAFIA BÁSICA:

PRICE, A. M. A. Implementação de Linguagens de Programação: Compiladores. 2 ed. Porto Alegre: Sagra-Luzzatto, 2001.

AHO, A. V.; Sethi, R.; Lam, Monica S. e Ullman, J. D. Compiladores: Princípios, Técnicas e Ferramentas. 2 ed, São Paulo: Prentice Hall, 2007.

BIBLIOGRAFIA COMPLEMENTAR:

HOPCROFT, J. E.; Ullman, J. D. Introduction to Automata Theory, Languages, and Computation. 1979. 418 f. Addison-Wesley Publishing Company, Massachusetts, EUA.

MENEZES, Paulo Fernando Blauth. Linguagens formais e autômatos. 1998. 165 f. Sagra-Luzzatto, 2a.Edição, Série livros didáticos, Porto Alegre.